

Deploying Tazama on AWS - Step-by-Step Instructions

Living document. This file records every command, output, and decision made during the first successful deployment of the three-stack Tazama system on AWS. It is intended to become a reproducible guide for community contributors who want to stand up the same environment.

Companion documents:

- `tazama-aws-deployment.md` - the architectural design and phase plan (the "what and why")
- `consolidate-full-stack.md` - the compose file and launcher script reference (the "how the stacks are assembled")

This document is the "what we actually did and what happened" log.

Prerequisites

Before starting, you need:

Requirement	Notes
Windows workstation	PowerShell 5.1+ or PowerShell 7. All scripts are <code>.ps1</code> .
AWS account	With an IAM user or role that has permissions to create EC2, VPC, Route 53, ALB, SSM, S3, and DynamoDB resources
Docker Desktop	For local testing only - not required for the cloud deploy itself
<code>full-stack-docker-tazama</code> repo	Cloned locally. All commands in this guide assume the repo root is your working directory unless stated otherwise.

All other required tools (AWS CLI, Git, OpenSSH, OpenTofu) are installed as part of Phase B.

Repository Layout

Three stacks are deployed across three EC2 instances. Each stack maps to a subfolder of the repo:

```
full-stack-docker-tazama/
├─ core/           → Server A - tazama-core (NATS, PostgreSQL, Valkey, TMS, rules, TP, auth, relay)
├─ extensions/     → Server B - tazama-extensions (OpenSearch, CMS, TCS, TRS, SFTP)
└─ biar/           → Server C - tazama-biar (NiFi, Solr, Tika, Apache Ozone, JupyterHub)
```

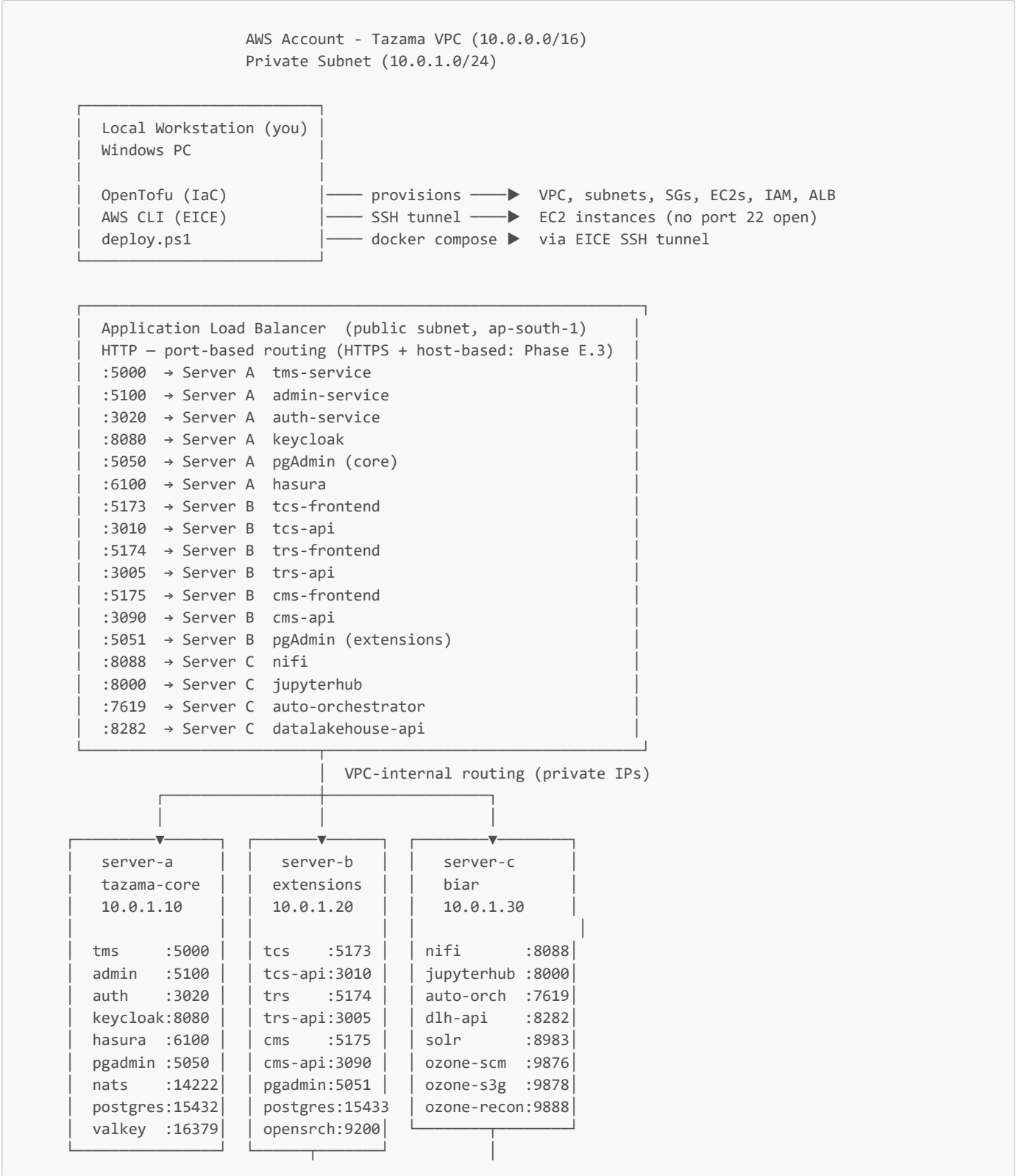
The IaC and deploy scripts live in a subfolder that must be created as part of Phase C:

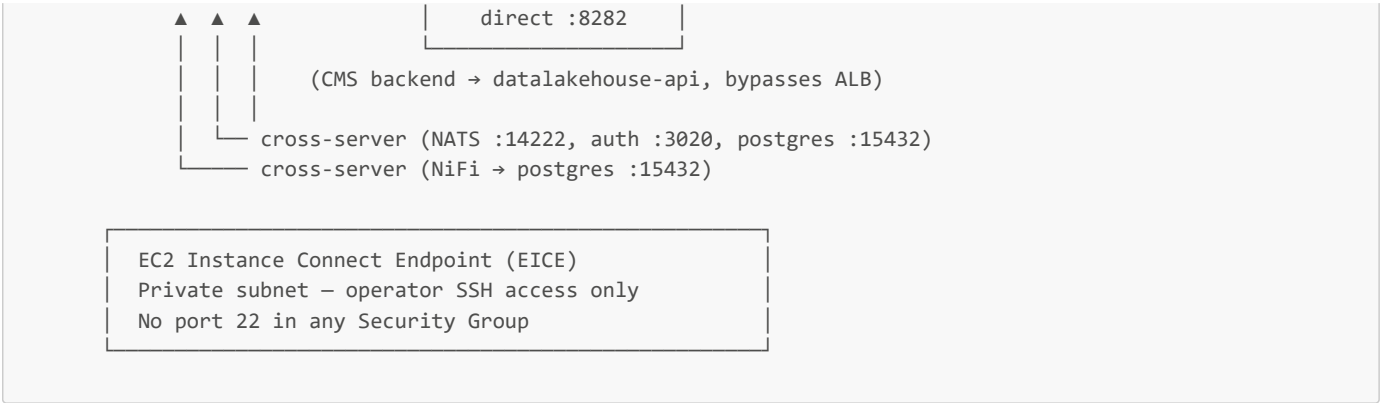
```
full-stack-docker-tazama/
├─ infra/
│   └─ aws/
│       ├── main.tf
│       ├── variables.tf
│       ├── outputs.tf
│       ├── terraform.tfvars           ← gitignored - operator-set values
│       ├── terraform.tfvars.example  ← committed - template for operators
│       ├── modules/
│       │   ├── vpc/
│       │   ├── security-groups/
│       │   ├── ec2/
│       │   └─ dns/
│       └─ templates/
│           ├── bootstrap.sh.tpl
│           ├── env-core.tpl
│           ├── env-extensions.tpl
│           └─ env-biar.tpl
```

```
└─ scripts/
   ├── helpers.ps1
   ├── deploy-core.ps1
   ├── deploy-extensions.ps1
   ├── deploy-biar.ps1
   ├── deploy.ps1
   └── teardown.ps1
```

Architecture Summary

Three private EC2 instances sit behind an Application Load Balancer. No instance has a public IP or any port 22 open. Operator SSH access uses the AWS EC2 Instance Connect Endpoint (EICE). All user traffic reaches services via the ALB.





Cross-server communication uses private DNS (Route 53 private hosted zone `tazama.internal`):

DNS name	Private IP	Stack
core.tazama.internal	10.0.1.10	Server A
extensions.tazama.internal	10.0.1.20	Server B
biar.tazama.internal	10.0.1.30	Server C

Phase A: Compose File Changes Required Before AWS Deployment

These are code changes to the `full-stack-docker-tazama` repo that must be applied before the stacks can run correctly on separate machines. On a single machine (local dev), Docker networks span all stacks automatically - on separate EC2 instances they do not.

All Phase A items are complete. They are documented here so community contributors understand what was changed and why.

A.1 - Remove cross-machine Docker network reference from biar compose

Problem: `biar/docker-compose.biar.infrastructure.yaml` declared an external network `tazama-core_default`. This works when all stacks run on one machine. On separate EC2 instances, the Docker network does not span machines and the biar stack fails to start.

Fix: Removed the `tazama-core_default` external network declaration and all per-service `networks:` references from the biar compose file. Docker auto-creates `tazama-biar_default`. NiFi reaches Server A services via `${SERVER_A_HOST}:<exterior-port>` env vars set in Parameter Contexts - no compose-level network join required.

Files changed:

- `biar/docker-compose.biar.infrastructure.yaml` - network block and per-service network references removed

A.2 - Update tazama-biar.bat guard check

Problem: `tazama-biar.bat` guarded against running unless `tazama-core` was up, using `docker compose -p tazama-core ps`. On Server C, `tazama-core` does not exist as a Docker project - this check always fails.

Fix: Replaced the Docker project existence check with a TCP reachability check against `${SERVER_A_HOST}:14222` (NATS exterior port read from `.env`). Works for both single-machine and multi-host deployments.

Files changed:

- `biar/tazama-biar.bat`

A.3 - Add SERVER_B_HOST to biar/.env

Problem: `biar/.env` only had `SERVER_A_HOST`. Any biar service that needs to reach Server B (OpenSearch on `:9200`, Server B PostgreSQL on `:15433`) had no variable for the host.

Fix: Added `SERVER_B_HOST=host.docker.internal` as the local default. The deploy script patches this to `extensions.tazama.internal` for AWS.

Files changed:

- `biar/.env`

A.4 - Verify and document core exterior port publishing

Finding: `core/docker-compose.base.override.yaml` publishes the three exterior ports that cross-stack services depend on: NATS :14222, PostgreSQL :15432, Valkey :16379. This file is always position 2 in every compose chain across all four deployment types (hub, dev, full, multitenant). Without it in the chain, Server B and Server C cannot reach Server A's services.

Action: Extracted the full compose chain matrix from `tazama-core.bat` and documented it in the deployment plan (Appendix I). Both hub (DockerHub images) and dev (GitHub source build) chains for AWS are captured.

No files changed in the stack. Documentation only.

A.5 - Parameterise CORS origins in extensions env files

Problem: `extensions/env/deapi.env`, `dems.env`, and `trs.env` had CORS and allowed-origins values hardcoded to `http://localhost:...`. On AWS, the browser-facing frontends are served from Server B - `localhost` is not correct.

Fix:

- Added `SERVER_B_HOST=localhost` to `extensions/.env` (alongside existing `SERVER_A_HOST=host.docker.internal`)
- Changed `CORS_ORIGINS=http://localhost:5173` → `CORS_ORIGINS=http://${SERVER_B_HOST}:5173` in `deapi.env` and `dems.env`
- Changed `ALLOWED_ORIGINS=http://localhost:5174` → `ALLOWED_ORIGINS=http://${SERVER_B_HOST}:5174` in `trs.env`

The deploy script patches `SERVER_B_HOST=extensions.tazama.internal` for AWS.

Phase G note: When ALB subdomains go live (e.g. `https://tcs.<your-zone>`) the CORS origins will also need the ALB subdomain added. See Phase G for details.

Files changed:

- `extensions/.env`
- `extensions/env/deapi.env`
- `extensions/env/dems.env`
- `extensions/env/trs.env`

A.6 - Re-enable OpenSearch security plugin (pending)

Problem: `extensions/docker-compose.extensions.infrastructure.yaml` runs OpenSearch with `DISABLE_SECURITY_PLUGIN=true`. Acceptable for local dev; unacceptable in a VPC where port 9200 is reachable from other instances.

Required changes:

- Remove `DISABLE_SECURITY_PLUGIN=true` from the `opensearch-node1` service environment in `extensions/docker-compose.extensions.infrastructure.yaml`
- Add `OPENSEARCH_INITIAL_ADMIN_PASSWORD` sourced from SSM at deploy time
- Update all services referencing OpenSearch (`cms-backend`, `tcs-backend`, `trs-backend`) to use the new credentials via SSM-injected env vars

Status: Pending - will be recorded here when executed.

A.7 - Replace published default credentials with SSM-sourced secrets (partially implemented)

Problem: All three stacks ship with credentials committed in plaintext in the public GitHub repository. These include database passwords (`unused`), admin passwords (`password`, `admin123456789`), API secrets, and service credentials spanning at least 15 variables across `core/`, `extensions/`, and `biar/`. While the VPC private subnet prevents direct internet access to most services, published credentials enable trivial lateral movement after any initial foothold and leave the ALB-accessible NiFi UI directly exposed to credential-stuffing attacks.

What has been implemented:

The deploy scripts now accept a `-Password` parameter. At deploy time the value is built into an in-memory overlay (never written to a committed file) and applied via `Set-RemoteEnvOverlay` to all PostgreSQL and Keycloak admin credential variables on Server A and Server B:

- `deploy-core.ps1 -Password '<pw>'` patches `core/.env` and all `core/env/` service env files on Server A: `POSTGRES_PASSWORD`, `RAW_HISTORY_DATABASE_PASSWORD`, `CONFIGURATION_DATABASE_PASSWORD`, `EVENT_HISTORY_DATABASE_PASSWORD`, `EVALUATION_DATABASE_PASSWORD`, `NEXT_PUBLIC_PG_PASSWORD`, `KEYCLOAK_ADMIN_PASSWORD`
- `deploy-extensions.ps1 -Password '<pw>'` patches `extensions/.env` and all `extensions/env/` service env files on Server B: `POSTGRES_PASSWORD`, `DB_PASSWORD`, `SPRING_DATASOURCE_PASSWORD`, `CONFIGURATION_DATABASE_PASSWORD`
- `deploy.ps1 -Password '<pw>'` passes the value through to both scripts above
- `env-core.tpl` was deleted (it previously committed only hardcoded credentials); `env-extensions.tpl` had its credential lines removed (DNS/URL overrides remain)

What is still pending (SSM-sourced secrets for remaining credentials):

The approach described below covers the remaining 10+ credentials (Redis, CouchDB, TRS crypto key, Hasura, pgAdmin, etc.) that the `-Password` param does not address. These are the eventual target; until then those services continue to use the published defaults.

Eventual approach:

SSM Parameter Store is already used for `GH_TOKEN` (B.9). The same pattern extends to all service credentials: secrets are stored in SSM before deployment, fetched by the deploy scripts at deploy time using `aws ssm get-parameter --with-decryption`, and written to the correct env files on each instance via the existing `Set-RemoteEnvOverlay` mechanism. No secret ever touches a committed file.

Several services use different variable names for the same underlying credential — for example, the PostgreSQL core password appears as `POSTGRES_PASSWORD` in the server container, `DB_PASSWORD` in some clients, and `RAW_HISTORY_DATABASE_PASSWORD` / `CONFIGURATION_DATABASE_PASSWORD` / `EVENT_HISTORY_DATABASE_PASSWORD` in others. The overlay templates map one SSM value to all of them, so the operator sets one password in SSM and it propagates consistently.

Local dev follow-on: The same consolidation is worth doing in the repo itself — adding canonical password variables to each stack's root `.env` and changing service env files to reference `${POSTGRES_PASSWORD}` etc., so a developer changes one line rather than hunting through a dozen files. This is a follow-on refactor, not part of the AWS deployment work.

Canonical SSM parameters:

SSM parameter	Protects	Notes
<code>/tazama/postgres_core_password</code>	PostgreSQL — Server A	Server container + all <code>core/env/</code> clients
<code>/tazama/postgres_extensions_password</code>	PostgreSQL — Server B (CMS)	Server container + <code>extensions/env/</code> clients
<code>/tazama/keycloak_admin_password</code>	Keycloak admin console	<code>core/env/keycloak.env</code>
<code>/tazama/hasura_admin_secret</code>	Hasura GraphQL admin	<code>core/docker-compose.utils.hasura.yaml</code>
<code>/tazama/pgadmin_password</code>	pgAdmin web UI	Both <code>core/env/pgadmin.env</code> and <code>extensions/env/pgadmin.env</code>
<code>/tazama/redis_password</code>	Redis/Valkey	<code>extensions/env/deapi.env</code> , <code>dems.env</code>
<code>/tazama/couchdb_password</code>	CouchDB admin	<code>extensions/env/cms.env</code> — username <code>simon</code> should also be replaced
<code>/tazama/auth_client_secret</code>	Auth service OAuth client	<code>core/env/auth-service.env</code>
<code>/tazama/trs_crypto_key</code>	TRS signing key	<code>extensions/env/trs.env</code> — use 32+ random characters
<code>/tazama/relay_auth_password</code>	REST relay auth	<code>core/env/rs-rest.env</code>
<code>/tazama/cms_auth_admin_password</code>	CMS Tazama auth admin	<code>extensions/env/cms.env</code>

SSM parameter	Protects	Notes
/tazama/cms_flowable_password	CMS Flowable engine	extensions/env/cms.env
/tazama/nifi_admin_password	NiFi single-user admin	biar/docker-compose.biar.infrastructure.yaml — NiFi requires min 12 chars
/tazama/ozone_s3g_secret_key	Apache Ozone S3G	biar/docker-compose.biar.infrastructure.yaml
/tazama/opensearch_password	OpenSearch admin	extensions/env/cms.env, tcs.env, trs.env — see also A.6
/tazama/sftp_password	SFTP server user	Special case — see note below

Mapping — SSM parameter → env variables patched:

SSM parameter	Variable name(s)	Target file(s)
/tazama/postgres_core_password	POSTGRES_PASSWORD	core/docker-compose.base.infrastructure.yaml
	POSTGRES_PASSWORD, DB_PASSWORD	core/env/ service env files (deapi, dems)
	RAW_HISTORY_DATABASE_PASSWORD, CONFIGURATION_DATABASE_PASSWORD, EVENT_HISTORY_DATABASE_PASSWORD, EVALUATION_DATABASE_PASSWORD, NEXT_PUBLIC_PG_PASSWORD	core/env/ service env files (admin, tms, tp, rules, tadp, etc.)
/tazama/postgres_extensions_password	POSTGRES_PASSWORD	extensions/docker-compose.extensions.infrastructure.yaml, extensions/.env, extensions/env/tcs.env
	POSTGRES_PASSWORD, SPRING_DATASOURCE_PASSWORD	extensions/env/cms.env
/tazama/keycloak_admin_password	KEYCLOAK_ADMIN_PASSWORD	core/env/keycloak.env
/tazama/hasura_admin_secret	HASURA_GRAPHQL_ADMIN_SECRET	core/docker-compose.utils.hasura.yaml
/tazama/pgadmin_password	PGADMIN_DEFAULT_PASSWORD	core/env/pgadmin.env, extensions/env/pgadmin.env
/tazama/redis_password	REDIS_PASSWORD	extensions/env/deapi.env, extensions/env/dems.env
/tazama/couchdb_password	COUCHDB_PASSWORD	extensions/env/cms.env
/tazama/auth_client_secret	CLIENT_SECRET	core/env/auth-service.env
/tazama/trs_crypto_key	CRYPTO_SECRET_KEY	extensions/env/trs.env
/tazama/relay_auth_password	AUTH_PASSWORD	core/env/rs-rest.env
/tazama/cms_auth_admin_password	TAZAMA_AUTH_ADMIN_PASSWORD	extensions/env/cms.env
/tazama/cms_flowable_password	FLOWABLE_PASSWORD	extensions/env/cms.env
/tazama/nifi_admin_password	SINGLE_USER_CREDENTIALS_PASSWORD	biar/docker-compose.biar.infrastructure.yaml
/tazama/ozone_s3g_secret_key	OZONE-SITE.XML_ozone.s3g.secret.key	biar/docker-compose.biar.infrastructure.yaml
/tazama/opensearch_password	OPENSEARCH_PASSWORD, OPENSEARCH_INITIAL_ADMIN_PASSWORD	extensions/env/cms.env, tcs.env, trs.env
/tazama/sftp_password	SFTP_PASSWORD_CONSUMER, SFTP_PASSWORD_PRODUCER	extensions/env/tcs.env — see note below

SFTP special case: The SFTP server user is defined in the `command:` line of the `atmoz/sftp` container in `extensions/docker-compose.extensions.infrastructure.yaml` as the literal string `user:password:1001`. This cannot be overridden by env var

substitution without first changing the compose file to `${SFTP_USER}:${SFTP_PASSWORD}:1001`. That compose file change is a prerequisite for the overlay. Additionally, the `SFTP_PASSWORD_CONSUMER` and `SFTP_PASSWORD_PRODUCER` values in `tcs.env` use a `hash:salt` format specific to the SFTP server's authentication scheme — replacement values must be generated in that same format.

Commands to store secrets in SSM (run once, before Phase D):

Generate strong random passwords before running these. PowerShell built-in:

```
Add-Type -AssemblyName System.Web
[System.Web.Security.Membership]::GeneratePassword(32, 4)
```

```
$region = "ap-south-1"
$profile = "tazama"

# PostgreSQL – Server A
aws ssm put-parameter --name /tazama/postgres_core_password --type SecureString `
  --value "<generate-strong-password>" `
  --description "PostgreSQL master password - Server A (core)" `
  --region $region --profile $profile

# PostgreSQL – Server B
aws ssm put-parameter --name /tazama/postgres_extensions_password --type SecureString `
  --value "<generate-strong-password>" `
  --description "PostgreSQL master password - Server B (extensions/CMS)" `
  --region $region --profile $profile

# Keycloak admin
aws ssm put-parameter --name /tazama/keycloak_admin_password --type SecureString `
  --value "<generate-strong-password>" `
  --description "Keycloak admin console password" `
  --region $region --profile $profile

# Hasura admin secret
aws ssm put-parameter --name /tazama/hasura_admin_secret --type SecureString `
  --value "<generate-strong-password>" `
  --description "Hasura GraphQL admin secret" `
  --region $region --profile $profile

# pgAdmin (applied to both servers)
aws ssm put-parameter --name /tazama/pgadmin_password --type SecureString `
  --value "<generate-strong-password>" `
  --description "pgAdmin web UI password" `
  --region $region --profile $profile

# Redis/Valkey
aws ssm put-parameter --name /tazama/redis_password --type SecureString `
  --value "<generate-strong-password>" `
  --description "Redis/Valkey auth password" `
  --region $region --profile $profile

# CouchDB
aws ssm put-parameter --name /tazama/couchdb_password --type SecureString `
  --value "<generate-strong-password>" `
  --description "CouchDB admin password" `
  --region $region --profile $profile

# Auth service OAuth client secret
aws ssm put-parameter --name /tazama/auth_client_secret --type SecureString `
  --value "<generate-strong-secret>" `
  --description "Auth service OAuth client secret" `
  --region $region --profile $profile

# TRS crypto signing key (32+ chars)
aws ssm put-parameter --name /tazama/trs_crypto_key --type SecureString `
```

```

--value "<generate-32-char-random-string>" `
--description "TRS transaction record signing key" `
--region $region --profile $profile

# REST relay auth password
aws ssm put-parameter --name /tazama/relay_auth_password --type SecureString `
--value "<generate-strong-password>" `
--description "REST relay service auth password" `
--region $region --profile $profile

# CMS auth admin password
aws ssm put-parameter --name /tazama/cms_auth_admin_password --type SecureString `
--value "<generate-strong-password>" `
--description "CMS Tazama auth admin password" `
--region $region --profile $profile

# CMS Flowable password
aws ssm put-parameter --name /tazama/cms_flowable_password --type SecureString `
--value "<generate-strong-password>" `
--description "CMS Flowable workflow engine password" `
--region $region --profile $profile

# NiFi admin password (minimum 12 characters enforced by NiFi)
aws ssm put-parameter --name /tazama/nifi_admin_password --type SecureString `
--value "<generate-12-char-minimum-password>" `
--description "NiFi single-user admin password" `
--region $region --profile $profile

# Apache Ozone S3G secret key
aws ssm put-parameter --name /tazama/ozone_s3g_secret_key --type SecureString `
--value "<generate-strong-secret>" `
--description "Apache Ozone S3 gateway secret key" `
--region $region --profile $profile

# OpenSearch admin password (also used by A.6 to re-enable the security plugin)
aws ssm put-parameter --name /tazama/opensearch_password --type SecureString `
--value "<generate-strong-password>" `
--description "OpenSearch admin password" `
--region $region --profile $profile

```

Verify all parameters were stored:

```

aws ssm get-parameters-by-path `
--path /tazama `
--region ap-south-1 `
--profile tazama `
--query "Parameters[*].{Name:Name,Type:Type,LastModified:LastModifiedDate}" `
--output table

```

Required changes to remaining deploy scripts and overlay templates (pending):

The deploy scripts now handle PostgreSQL and Keycloak credentials via `-Password`. The remaining credentials listed in the SSM table above still need SSM integration. The pattern is the same as what was implemented:

1. Fetch each secret at deploy time: `aws ssm get-parameter --name /tazama/<param> --with-decryption --query Parameter.Value --output text`
2. Build a combined overlay in-memory that includes the credential entries
3. Pass the combined overlay to `Set-RemoteEnvOverlay` -- the existing `sed`-based patching handles the rest

Status: PostgreSQL + Keycloak credentials - implemented. Remaining 10+ credentials - pending. Required before this deployment is safe for beta use.

Phase B: Local Tooling Setup

These steps are run once on the local workstation before any infrastructure is provisioned.

B.1 - Install AWS CLI v2

AWS CLI is required for account authentication, establishing EICE SSH tunnels to private EC2 instances, and managing secrets in SSM Parameter Store.

Command:

```
winget install Amazon.AWSCLI
```

Expected output:

```
Found AWS Command Line Interface v2 [Amazon.AWSCLI] Version x.x.x.x
This application is licensed to you by its owner.
...
Successfully installed
```

Verify (close and reopen the terminal first to pick up the updated PATH):

```
aws --version
```

Expected output:

```
aws-cli/2.x.x Python/3.x.x Windows/x.exe/AMD64
```

Actual output:

```
<!-- record actual output here -->
```

B.2 - Install Git

Git is needed locally to manage the repo and is also installed on the EC2 instances by the bootstrap script (for cloning the repo at deploy time).

Check if already installed:

```
git --version
```

If the command is not found:

```
winget install Git.Git
```

Verify:

```
git --version
```

Expected output:

```
git version 2.x.x.windows.x
```

Actual output:

```
<!-- record actual output here -->
```

B.3 - Verify OpenSSH client

OpenSSH is used by the deploy scripts to run remote commands on EC2 instances through the EICE tunnel. It ships with Windows 10/11 but may need enabling.

Check if already installed:

```
ssh -V
```

If the command is not found, enable the built-in Windows OpenSSH client:

```
Add-WindowsCapability -Online -Name OpenSSH.Client~~~~0.0.1.0
```

Verify:

```
ssh -V
```

Expected output:

```
OpenSSH_for_Windows_9.5p2, LibreSSL 3.8.2
```

Actual output:

```
<!-- record actual output here -->
```

B.4 - Install OpenTofu

Why OpenTofu instead of Terraform? OpenTofu is the open-source fork of Terraform, created after HashiCorp moved Terraform to the Business Source License (BSL) in August 2023. OpenTofu is governed by the Linux Foundation under MPL-2.0 with no commercial-use restrictions. It is a drop-in replacement - same HCL syntax, same AWS provider, same state format - and installs identically. The CLI command is **tofu** (not **terraform**).

Command:

```
winget install OpenTofu.Tofu
```

Expected output:

```
Found OpenTofu [OpenTofu.Tofu] Version 1.11.6
This application is licensed to you by its owner.
Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
```

```
Downloading https://github.com/opentofu/opentofu/releases/download/v1.11.6/tofu_1.11.6_windows_amd64.zip
████████████████████████████████████████ 33.0 MB / 33.0 MB
Successfully verified installer hash
Extracting archive...
Successfully extracted archive
Starting package install...
Path environment variable modified; restart your shell to use the new value.
Command line alias added: "tofu"
Successfully installed
```

Verify:

```
tofu version
```

Expected output:

```
OpenTofu v1.x.x
on windows_amd64
```

Actual output:

```
<!-- record actual output here -->
```

B.5 - Clone the repository

```
git clone https://github.com/tazama-lf/full-stack-docker-tazama.git
cd full-stack-docker-tazama
```

All subsequent Phase B-E commands assume this directory as the working directory unless stated otherwise.

Actual output:

```
<!-- record actual output here -->
```

B.6 - Configure AWS CLI profile**Step 1: Create an IAM user and generate access keys**

You need an AWS IAM user with programmatic access. If you already have access keys, skip to Step 2.

1. Sign in to the [AWS Management Console](#)
2. In the search bar at the top, type **IAM** and open the IAM service
3. In the left sidebar, click **Users**, then click **Create user**
4. Enter a username (e.g., `tazama-deploy`) and click **Next**
5. On the permissions page, select **Attach policies directly**
6. Search for and attach the following AWS managed policies. These are the AWS services OpenTofu will provision and the deploy scripts will interact with - the user needs permission to create, modify, and destroy resources in each:

Policy	Why it's needed
Policy	Why it's needed
---	---
<code>AmazonEC2FullAccess</code>	Create and manage the three EC2 instances, key pairs, and the EC2 Instance Connect Endpoint (EICE) for SSH access
<code>AmazonVPCFullAccess</code>	Create the VPC, public and private subnets, Internet Gateway, NAT Gateway, and route tables
<code>AmazonRoute53FullAccess</code>	Create the private hosted zone (<code>tazama.internal</code>) and DNS A records for cross-server communication; also creates the public hosted zone and DNS validation records for the ACM certificate (Option 3)
<code>ElasticLoadBalancingFullAccess</code>	Create and configure the Application Load Balancer, target groups, and listener rules
<code>AmazonSSMFullAccess</code>	Store and retrieve secrets (GH_TOKEN, passwords) in SSM Parameter Store; also required for the EC2 Instance Connect Endpoint
<code>AmazonS3FullAccess</code>	Create the S3 bucket that stores OpenTofu state files
<code>AmazonDynamoDBFullAccess</code>	Create the DynamoDB table used for OpenTofu state locking (prevents concurrent applies)
<code>IAMFullAccess</code>	Create IAM instance profiles and roles so EC2 instances can read SSM parameters on startup

There is no AWS-managed policy for ACM certificate management or EICE tunnel access — these permissions are **not** covered by any AWS-managed policy listed above. Create customer-managed policies and attach them the same way:

ACM certificate policy (required for Phase E Option 3 custom domain):

```
$policy = '{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Action":
["acm:*"],"Resource":"*"}]}'
$policy | Out-File "$env:TEMP\acm-policy.json" -Encoding ASCII
$policyArn = (& "C:\Program Files\Amazon\AWSCLI2\aws.exe" iam create-policy `
  --policy-name TazamaACMAccess `
  --policy-document "file://$env:TEMP/acm-policy.json" `
  --description "Allows ACM certificate management for Tazama deploy user" `
  --profile tazama `
  --query Policy.Arn --output text)
& "C:\Program Files\Amazon\AWSCLI2\aws.exe" iam attach-user-policy `
  --user-name tazama-deploy `
  --policy-arn $policyArn `
  --profile tazama
```

EICE tunnel policy (required for all deploy scripts):

There is no AWS-managed policy for EICE tunnel access — `ec2-instance-connect:OpenTunnel` is **not** covered by `AmazonEC2FullAccess` or any AWS-managed policy. Create a customer-managed policy and attach it the same way as the policies above:

```
# Create the customer-managed policy (once per account)
$policy = '{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Action":["ec2-instance-
connect:OpenTunnel","ec2-instance-connect:SendSSHPublicKey"],"Resource":"*"}]}'
$policy | Out-File "$env:TEMP\eice-policy.json" -Encoding ASCII
$policyArn = (& "C:\Program Files\Amazon\AWSCLI2\aws.exe" iam create-policy `
  --policy-name TazamaEICEAccess `
  --policy-document "file://$env:TEMP/eice-policy.json" `
  --description "Allows EICE SSH tunnel (OpenTunnel) for Tazama deploy user" `
  --profile tazama `
  --query Policy.Arn --output text)
```

```
# Attach it to the user
& "C:\Program Files\Amazon\AWSCLIV2\aws.exe" iam attach-user-policy `
  --user-name tazama-deploy `
  --policy-arn $policyArn `
  --profile tazama
```

After this, `TazamaEICEAccess` appears in the same **Attached policies** list as `AmazonEC2FullAccess` etc. Without it, all deploy-script SSH connections fail silently with `AccessDeniedException`, causing `Wait-Bootstrap` to time out.

7. Click **Next**, review, then click **Create user**
8. Click on the newly created user, then open the **Security credentials** tab
9. Scroll to **Access keys** and click **Create access key**
10. Select **Command Line Interface (CLI)** as the use case, tick the confirmation checkbox, and click **Next**

"Alternatives recommended" message: AWS may display a banner here suggesting `aws login` (IAM Identity Center / SSO) or AWS CloudShell instead. These are AWS's preferred options for teams with an existing SSO setup. For this deployment:

- **aws login / IAM Identity Center** - requires prior SSO configuration in your AWS organisation. If your account already has IAM Identity Center set up, use it. If not, it adds significant setup overhead for a one-off deployment.
- **AWS CloudShell** - a browser-based terminal inside the AWS Console. It cannot run OpenTofu or the PowerShell deploy scripts, so it doesn't work here.

Proceed with the static access key. The message is a recommendation, not a blocker. Tick the confirmation checkbox and continue.

11. In the **Description tag value** field, enter something that identifies where this key is used, e.g. `tazama-deploy-local`. Then click **Create access key**
12. **Copy both the Access Key ID and the Secret Access Key now** - the secret is only shown once. Store them in a password manager.

Security note: These credentials grant broad access to your AWS account. Treat them like a password. Never commit them to a repository. Rotate them after the deployment is complete if they were created for this purpose only.

Step 2: Run `aws configure`

Command:

```
aws configure --profile tazama
```

You will be prompted for:

- `AWS Access Key ID` → paste the Access Key ID from Step 1
- `AWS Secret Access Key` → paste the Secret Access Key from Step 1
- `Default region name` → enter `ap-south-1`
- `Default output format` → enter `json`

The credentials are stored in `~/.aws/credentials` on your local machine (never leave the workstation).

Verify (confirm you are targeting the correct account):

```
aws sts get-caller-identity --profile tazama
```

Expected output:

```
{
  "UserId": "AIDAXXXXXXXXXXXXXXXXXX",
  "Account": "123456789012",
```

```
"Arn": "arn:aws:iam::123456789012:user/your-iam-user"
}
```

Actual output:

```
<!-- record actual output here -->
```

B.7 - Generate EC2 SSH key pair

This key pair is used by the deploy scripts to SSH through the EICE tunnel to EC2 instances. The private key is stored locally only - never committed.

Command: Paste the entire block at once. PowerShell will execute each statement as it recognises it as complete, showing `>>` continuation prompts for the multi-line `aws` command. The last line (`icacls`) will sit at the prompt waiting - press **Enter** once to run it.

```
# From the repo root
New-Item -ItemType Directory -Force -Path infra/aws | Out-Null
aws ec2 create-key-pair `
  --key-name tazama-aws `
  --key-type ed25519 `
  --query "KeyMaterial" `
  --output text `
  --profile tazama | Out-File -Encoding ascii infra/aws/tazama-aws.pem
# Restrict permissions (SSH client requires the key file is not world-readable)
icacls infra/aws/tazama-aws.pem /inheritance:r /grant:r "${env:USERNAME}:R"
```

Verify the file was created:

```
Test-Path infra/aws/tazama-aws.pem
```

Important: `infra/aws/tazama-aws.pem` must be in `.gitignore`. Verify before committing anything.

Expected output:

```
True
```

Actual output:

```
<!-- record actual output here -->
```

B.8 - Create S3 bucket and DynamoDB table for OpenTofu state

OpenTofu state is stored in S3 with DynamoDB for state locking. These resources are created once manually - they must exist before `tofu init` can run.

Replace `<account-id>` with your 12-digit AWS Account ID - this is the `Account` value from the `aws sts get-caller-identity` output in B.6 (e.g. `123456789012`). It is not the `UserId` or the `Access Key ID`. Using it as a suffix makes the S3 bucket name globally unique.

```
# Create S3 bucket (bucket name must be globally unique)
aws s3 mb s3://tazama-tofu-state-<account-id> `
  --region ap-south-1 `
  --profile tazama
```

```
# Enable versioning (allows state rollback)
aws s3api put-bucket-versioning `
  --bucket tazama-tofu-state-<account-id> `
  --versioning-configuration Status=Enabled `
  --profile tazama

# Enable server-side encryption
# (PowerShell strips double quotes from strings passed to external executables regardless of quoting
# approach;
# the reliable workaround for AWS CLI on Windows is to pass JSON via a temp file)
'{"Rules":[{"ApplyServerSideEncryptionByDefault":{"SSEAlgorithm":"AES256"}}}]' | Out-File
"$env:TEMP\enc.json" -Encoding ASCII
aws s3api put-bucket-encryption `
  --bucket tazama-tofu-state-<account-id> `
  --server-side-encryption-configuration "file://$env:TEMP/enc.json" `
  --profile tazama

# Block public access
aws s3api put-public-access-block `
  --bucket tazama-tofu-state-<account-id> `
  --public-access-block-configuration `
    "BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true" `
  --profile tazama

# Create DynamoDB table for state locking
aws dynamodb create-table `
  --table-name tazama-tofu-locks `
  --attribute-definitions AttributeName=LockID,AttributeType=S `
  --key-schema AttributeName=LockID,KeyType=HASH `
  --billing-mode PAY_PER_REQUEST `
  --region ap-south-1 `
  --profile tazama
```

Verify:

```
aws s3 ls s3://tazama-tofu-state-<account-id> --profile tazama
aws dynamodb describe-table --table-name tazama-tofu-locks --region ap-south-1 --profile tazama --query
"Table.TableStatus"
```

Expected DynamoDB status output: "ACTIVE"**Actual outputs:**

```
# S3 bucket:
make_bucket: tazama-tofu-state-381491982122

# Versioning: (no output = success)

# Encryption: (no output = success)

# Public access block: (no output = success)

# DynamoDB verify:
"ACTIVE"
```

B.9 - Store GH_TOKEN in SSM Parameter Store

The GitHub PAT is required for pulling from GitHub Container Registry and for source builds. It is stored in SSM as a **SecureString** - never written to any file in the repo or on the instances.

Required PAT scopes:

- `read:packages` - pull images from GitHub Container Registry (ghcr.io)
- `repo` - for GitHub source builds (dev build type only)

```
aws ssm put-parameter `
  --name /tazama/gh_token `
  --type SecureString `
  --value "<your-github-pat>" `
  --description "GitHub PAT for Tazama image pulls and source builds" `
  --region ap-south-1 `
  --profile tazama
```

Verify (confirms the parameter exists without revealing the value):

```
aws ssm get-parameter `
  --name /tazama/gh_token `
  --region ap-south-1 `
  --profile tazama `
  --query "Parameter.{Name:Name,Type:Type,LastModifiedDate:LastModifiedDate}"
```

Actual output:

```
<!-- record actual output here -->
```

B.10 - Confirm region and note constraints

Region selected: `ap-south-1` (Mumbai)

Rationale: Best latency balance across East Africa (~60 ms), Middle East (~30 ms), and South-East Asia (~70 ms) - the primary beta user regions. Lower cost than `af-south-1` or `me-central-1`. Full `t3` instance family availability.

Confirm `t3` availability in region:

```
aws ec2 describe-instance-type-offerings `
  --filters "Name=instance-type,Values=t3.xlarge,r5.2xlarge" `
  --region ap-south-1 `
  --profile tazama `
  --query "InstanceTypeOfferings[*].{Type:InstanceType,AZ:Location}" `
  --output table
```

Confirm Amazon Linux 2023 AMI ID:

```
aws ec2 describe-images `
  --owners amazon `
  --filters `
    "Name=name,Values=al2023-ami-*-x86_64" `
    "Name=state,Values=available" `
  --query "sort_by(Images, &CreationDate)[-1].{ID:ImageId,Name:Name,Date:CreationDate}" `
  --region ap-south-1 `
  --profile tazama `
  --output table
```

Record the AMI ID - it goes into `terraform.tfvars` in Phase C.

Actual outputs:


```
<!-- record actual output here -->
```

Phase C: OpenTofu Infrastructure

This phase creates all the HCL files that describe the AWS infrastructure, initialises the S3 backend, and runs a plan to verify correctness before anything is created.

Architecture summary

Resource	Detail
VPC	10.0.0.0/16
Public subnets	10.0.0.0/24 (ap-south-1a) . 10.0.2.0/24 (ap-south-1b)
Private subnet	10.0.1.0/24 (ap-south-1a) - all three EC2 instances
NAT Gateway	In public subnet 1 - lets instances reach internet
EICE endpoint	In private subnet - SSH without port 22 open to internet
Server A (core)	10.0.1.10 . t3.xlarge . 50 GB gp3
Server B (extensions)	10.0.1.20 . t3.xlarge . 50 GB gp3
Server C (biar)	10.0.1.30 . r5.2xlarge . 100 GB gp3
DNS	Route 53 private zone tazama.internal
ALB	Optional — see Phase E
AMI	Amazon Linux 2023 (fetched dynamically - always latest)

C.1 Folder structure

The IaC files live inside the cloned repo. All paths are relative to full-stack-docker-tazama\infra\aws\.

```
infra/aws/
├─ main.tf                # root module - assembles all sub-modules
├─ variables.tf
├─ outputs.tf
├─ terraform.tfvars.example  # committed; copy → terraform.tfvars (gitignored)
├─ backend.conf.example      # committed; copy → backend.conf (gitignored)
├─ modules/
│   ├── vpc/                main.tf  variables.tf  outputs.tf
│   ├── security-groups/    "
│   ├── ec2/                 "
│   ├── dns/                 "
│   ├── alb/                 "          # Phase E: public sandbox option
│   └── dns-public/          "          # Phase G: custom domain (optional)
└─ templates/
    ├── bootstrap.sh.tpl     # user_data - Docker install, repo clone, GH_TOKEN
    ├── env-extensions.tpl    # Phase D sed overlay for extensions/.env
    └── env-biar.tpl          # Phase D sed overlay for biar/.env
```

.gitignore already excludes the key pair file (*.pem), terraform.tfvars, and the new backend.conf.

C.2 modules/vpc/

Three files define the network layer.

What gets created:

- VPC (10.0.0.0/16, DNS support + hostnames enabled)
- Internet Gateway
- Two public subnets across two AZs (required for ALB)
- One private subnet in AZ-1 (all EC2 instances live here)
- Elastic IP + NAT Gateway in public subnet 1 (instances need internet for package installs and ghcr.io pulls)
- Public and private route tables
- EICE security group (outbound TCP 22 to private subnet only - no inbound)
- EC2 Instance Connect Endpoint in the private subnet

Key decision - EICE vs bastion host:

Standard SSH requires port 22 open to 0.0.0.0/0 or a bastion in a public subnet. EC2 Instance Connect Endpoint (aws_ec2_instance_connect_endpoint) provides browser-free, port-22-free SSH tunnelling authenticated via IAM. The EICE SG allows only outbound TCP 22 to 10.0.1.0/24; each instance SG allows port 22 only from that EICE SG.

Files created:

- infra/aws/modules/vpc/variables.tf
- infra/aws/modules/vpc/main.tf
- infra/aws/modules/vpc/outputs.tf

C.3 modules/security-groups/

Four security groups. EC2 instances have **no internet-facing inbound rules** — all user traffic enters via the ALB, all operator SSH enters via EICE.

ALB routing mode: The ALB currently uses **port-based HTTP routing** — each service is reachable at http://<alb-dns>:<port>. Port 443 is pre-configured in the ALB SG for the future custom-domain HTTPS upgrade (Phase F), at which point all service ports except 443 can be removed from the ALB SG and traffic will route via SNI host header rules.

sg-tazama-alb (Application Load Balancer)

Direction	Port(s)	Protocol	Source	Notes
Inbound	443	TCP	0.0.0.0/0	HTTPS — Phase F custom domain (pre-configured)
Inbound	5000	TCP	0.0.0.0/0	TMS API
Inbound	5050–5051	TCP	0.0.0.0/0	pgAdmin (Server A + B)
Inbound	5100	TCP	0.0.0.0/0	Admin API
Inbound	3020	TCP	0.0.0.0/0	Auth Service
Inbound	8080	TCP	0.0.0.0/0	Keycloak
Inbound	6100	TCP	0.0.0.0/0	Hasura
Inbound	3005–3090	TCP	0.0.0.0/0	TRS / TCS / CMS backends
Inbound	5173–5175	TCP	0.0.0.0/0	TCS / TRS / CMS frontends
Inbound	8088	TCP	0.0.0.0/0	NiFi UI
Outbound	All	All	0.0.0.0/0	Routing to EC2 target groups

JupyterHub (:8000), Automation Orchestrator (:7619), and Datalakehouse API (:8282) have ALB target groups and listeners, but their ports are **not** open in the ALB SG — they are accessed via SSH tunnel (Phase E.1). Add them to the ALB SG when exposing them publicly.

sg-tazama-server-a (Server A — tazama-core)

Direction	Port(s)	Protocol	Source	Notes
Inbound	5000	TCP	sg-tazama-alb	TMS API
Inbound	3001–3020	TCP	sg-tazama-alb	DEAPI / DEMS / Auth Service range

Direction	Port(s)	Protocol	Source	Notes
Inbound	5100	TCP	sg-tazama-alb	Admin API
Inbound	8080	TCP	sg-tazama-alb	Keycloak
Inbound	5050–5051	TCP	sg-tazama-alb	pgAdmin
Inbound	6100	TCP	sg-tazama-alb	Hasura
Inbound	0–65535	TCP	10.0.1.0/24	Cross-server (NATS :14222, PostgreSQL :15432, Valkey :16379, etc.)
Inbound	22	TCP	sg-tazama-eice	SSH via EICE endpoint only
Outbound	All	All	0.0.0.0/0	Image pulls, GitHub builds

sg-tazama-server-b (Server B — tazama-extensions)

Direction	Port(s)	Protocol	Source	Notes
Inbound	3005–3090	TCP	sg-tazama-alb	TRS / TCS / CMS backends
Inbound	5173–5175	TCP	sg-tazama-alb	TCS / TRS / CMS frontends
Inbound	5051	TCP	sg-tazama-alb	pgAdmin (extensions)
Inbound	0–65535	TCP	10.0.1.0/24	Cross-server (OpenSearch :9200, PostgreSQL :15433, etc.)
Inbound	22	TCP	sg-tazama-eice	SSH via EICE endpoint only
Outbound	All	All	0.0.0.0/0	Image pulls, GitHub builds, Server A calls

sg-tazama-server-c (Server C — tazama-biar)

Direction	Port(s)	Protocol	Source	Notes
Inbound	8088	TCP	sg-tazama-alb	NiFi UI
Inbound	8000	TCP	sg-tazama-alb	JupyterHub
Inbound	7619	TCP	sg-tazama-alb	Automation Orchestrator
Inbound	8282	TCP	sg-tazama-alb	Datalakehouse API (via ALB / tunnel)
Inbound	8282	TCP	sg-tazama-server-b	Datalakehouse API (CMS backend direct call — bypasses ALB)
Inbound	22	TCP	sg-tazama-eice	SSH via EICE endpoint only
Outbound	All	All	0.0.0.0/0	Image pulls, calls to Server A + B

Solr (:8983), Ozone SCM (:9876), Ozone S3G (:9878), and Ozone Recon (:9888) are **internal-only** — no SG inbound rules; accessed exclusively via the SSH tunnel (`tunnel-server-c.ps1`).

`sg-tazama-eice` is a small separate security group attached to the EICE VPC endpoint itself. It has no inbound rules; outbound allows TCP port 22 to `10.0.1.0/24` only. Each instance SG allows port 22 from this EICE SG only — not from the internet.

Files created:

- [infra/aws/modules/security-groups/variables.tf](#)
- [infra/aws/modules/security-groups/main.tf](#)
- [infra/aws/modules/security-groups/outputs.tf](#)

C.4 `modules/ec2/`

A reusable module for one EC2 instance. All three servers are created from it.

Notable settings:

- `private_ip` - fixed address; ensures DNS records are stable across stop/start

- `root_block_device` - gp3, encrypted, `delete_on_termination = true`
- `metadata_options` - IMDSv2 required (`http_tokens = "required"`); prevents SSRF-based credential theft via the instance metadata endpoint
- `iam_instance_profile` - grants the instance SSM read-only access (for the bootstrap GH_TOKEN fetch); no credentials in `user_data`

Files created:

- [infra/aws/modules/ec2/variables.tf](#)
- [infra/aws/modules/ec2/main.tf](#)
- [infra/aws/modules/ec2/outputs.tf](#)

C.5 modules/dns/

Creates a Route 53 **private** hosted zone (`tazama.internal`) associated with the VPC, then one A record per server:

Hostname	IP
<code>core.tazama.internal</code>	<code>10.0.1.10</code>
<code>extensions.tazama.internal</code>	<code>10.0.1.20</code>
<code>biar.tazama.internal</code>	<code>10.0.1.30</code>

These names are used in `.env` overlays so container-to-container calls cross servers by name, not IP - the IP is fixed anyway, but names survive a `tofu destroy` → re-apply cycle.

Files created:

- [infra/aws/modules/dns/variables.tf](#)
- [infra/aws/modules/dns/main.tf](#)
- [infra/aws/modules/dns/outputs.tf](#)

C.6 Bootstrap script template

[infra/aws/templates/bootstrap.sh.tpl](#) is rendered by `templatefile()` in `main.tf` and passed as `user_data` to all three instances.

It runs once at first boot and:

1. Installs Docker CE from the Amazon Linux 2023 dnf repo
2. Installs the Docker Compose v2 plugin from GitHub releases
3. Clones the `tazama-lf/full-stack-docker-tazama` repo to `/home/ec2-user/full-stack-docker-tazama`
4. Fetches `GH_TOKEN` from SSM Parameter Store (`/tazama/gh_token`) using the instance's IAM role - no credentials in `user_data`
5. Writes `GH_TOKEN` to `/etc/environment` (available to all sessions)
6. Logs in to `ghcr.io` and copies the Docker credential store to `/home/ec2-user/.docker/` so the `ec2-user` account can pull images
7. Writes `/home/ec2-user/.bootstrap-complete` marker

Template variables:

Variable	Source	Default	Purpose
<code>\${region}</code>	<code>var.region</code> in <code>variables.tf</code>	<code>ap-south-1</code>	AWS region passed to <code>aws configure</code>
<code>\${repo_branch}</code>	<code>var.repo_branch</code> in <code>variables.tf</code>	<code>dev</code>	Git branch cloned at first boot

To deploy instances onto a different branch, set `repo_branch` in `terraform.tfvars` before running `tofu apply`:

```
repo_branch = "my-feature-branch"
```

Bash variables in the template use `$$${VAR}` (double dollar) which OpenTofu renders as `${VAR}` - standard bash parameter expansion in the output script.

C.7 .env overlay templates

These are static key=value files consumed by the Phase D deploy scripts. They override the local-dev `SERVER_*_HOST` values in `extensions/.env` and `biar/.env` with the Route 53 private DNS names.

`core/.env` has no cross-server host variables, so no overlay is needed.

Files created:

- [infra/aws/templates/env-extensions.tpl](#)
- [infra/aws/templates/env-biar.tpl](#)

`env-extensions.tpl` overrides include:

Variable	Committed default	Overlay value	Reason
<code>SERVER_A_HOST</code>	<code>10.0.1.10</code>	<code>core.tazama.internal</code>	Private DNS for cross-stack calls
<code>SERVER_B_HOST</code>	<code>10.0.1.20</code>	<code>extensions.tazama.internal</code>	Private DNS
<code>ADMIN_SERVICE_URL</code>	container-internal port	<code>http://core.tazama.internal:5100</code>	Correct admin port
<code>TRS_API_URL /</code> <code>TCS_API_URL /</code> <code>CMS_API_URL</code>	localhost defaults	Public ALB subdomains	Browser-facing VITE_vars
<code>SIMULATION_ENDPOINT /</code> <code>ADMIN_ENDPOINT</code>	localhost defaults	Public ALB subdomains	Browser-facing VITE_vars
<code>ALLOWED_ORIGINS /</code> <code>CORS_ORIGINS</code>	localhost	Public ALB subdomains	CORS
<code>GOLD_LAKEHOUSE_API_URL</code>	<code>http://\${SERVER_C_HOST}:8282</code> (placeholder, resolves to dev default at runtime)	<code>SERVER_C_HOST=biar.tazama.internal</code> set by this overlay	Server C datalakehouse-api

C.8 Root `main.tf` and `variables.tf`

[infra/aws/main.tf](#) ties everything together:

- Backend block is **intentionally empty** (`backend "s3" {}`); the S3 config is supplied via `backend.conf` at `tofu init` time (keeps account IDs out of committed HCL).
- IAM role + instance profile defined here (shared by all three instances).
- `data "aws_ami" "al2023"` fetches the newest Amazon Linux 2023 x86_64 image dynamically - no AMI ID to manually update.
- `locals { bootstrap = templatefile(...) }` renders the bootstrap script once and passes the same rendered string to all three `module "server_*` calls.

[infra/aws/variables.tf](#) - only `key_name` is required; everything else has a sensible default.

Key optional variables:

Variable	Default	Purpose
<code>repo_branch</code>	<code>dev</code>	Git branch cloned on EC2 instances at bootstrap. Override in <code>terraform.tfvars</code> to target a different branch for fresh deployments.
<code>instance_type_a /</code> <code>_b</code>	<code>t3.xlarge</code>	EC2 size for Server A and B
<code>instance_type_c</code>	<code>r5.2xlarge</code>	EC2 size for Server C (Ozone needs memory)

[infra/aws/terraform.tfvars.example](#)

- copy to `terraform.tfvars` (gitignored) and set at minimum `key_name = "tazama-aws"`.

[infra/aws/backend.conf.example](#)

- copy to `backend.conf` (gitignored) and set the bucket name with your account ID.

C.9 Root `outputs.tf`

[infra/aws/outputs.tf](#) surfaces the values the Phase D deploy scripts need: instance IDs, private IPs, and the EICE endpoint ID (used in the SSH `ProxyCommand`).

C.10 Prepare config files and run `tofu init`

Replace `<your-account-id>` in the command below with your 12-digit AWS Account ID - the `Account` value from the `aws sts get-caller-identity` output in B.6 (e.g. `123456789012`). It is not the `UserId` or the Access Key ID.

```
cd full-stack-docker-tazama\infra\aws

# 1. terraform.tfvars - only key_name is required
Copy-Item terraform.tfvars.example terraform.tfvars
# (the defaults are already correct for this deployment)

# 2. backend.conf - replace <your-account-id> in the command below with your
# 12-digit AWS Account ID (the Account value from B.6, e.g. 123456789012)
Copy-Item backend.conf.example backend.conf
(Get-Content backend.conf) -replace '<your-account-id>', '123456789012' |
Set-Content backend.conf

# 3. Initialise OpenTofu (downloads AWS provider, configures S3 backend)
tofu init -backend-config backend.conf
```

Expected output:

```
Initializing the backend...

Successfully configured the backend "s3"! OpenTofu will automatically
use this backend unless the backend configuration changes.
Initializing modules...
- alb in modules\alb
- dns in modules\dns
- dns_public in modules\dns-public
- ec2 in modules\ec2
- security_groups in modules\security-groups
- vpc in modules\vpc
...
OpenTofu has been successfully initialized!
```

Record actual output below once C.10 is run.

C.11 `tofu plan`

If you intend to use ALB access (Phase E.2 or E.3), include `alb.tfvars` in the plan now:

```
# Recommended - ALB included upfront (Phase E.2 / E.3)
tofu plan -var-file terraform.tfvars -var-file alb.tfvars -out tfplan

# ALB-free - SSH tunnelling only (Phase E.1)
tofu plan -var-file terraform.tfvars -out tfplan
```

Why include the ALB at plan time? Keycloak requires its public hostname (`KC_HOSTNAME`) to be configured before the container first starts. The `deploy-core.ps1` script reads the ALB DNS name from `tofu output` and injects `KEYCLOAK_HOSTNAME` into the remote `core/.env` automatically — but only if the ALB has already been applied. Applying the ALB after deployment requires a manual Keycloak container restart (see E.2.1).

The `-out tfplan` flag saves the plan to a file. When you run `tofu apply` in C.12 you pass `tfplan` directly - OpenTofu then executes exactly the plan you reviewed, with no risk of drift between plan and apply.

Review the plan carefully. Expected resource count for core-only is roughly 40-45 resources (VPC, subnets, IGW, NAT GW, EIP, route tables, SGs, EICE endpoint, IAM role + attachment + profile, AMI data source, 3 × EC2, Route 53 zone + 3 records). Adding `alb.tfvars` adds roughly 33 more (1 ALB, 17 target groups, 17 port-based HTTP listeners, SG rules).

Record Plan: X to add, 0 to change, 0 to destroy. line once C.11 is run.

C.12 `tofu apply`

```
tofu apply tfplan
```

This creates all the AWS infrastructure defined in the plan. Because you saved the plan with `-out tfplan` in C.11, OpenTofu applies exactly what you reviewed -- no re-planning, no drift.

Expect this to take 3-8 minutes (5-10 min with ALB). The EC2 Instance Connect Endpoint is usually the slowest resource to provision (~5 min).

Record Apply complete! Resources: X added, 0 changed, 0 destroyed. line once C.12 is run.

Phase D: Deployment Scripts

All scripts live in `infra/aws/scripts/`. **No modification is required** - the region (`ap-south-1`), AWS profile (`tazama`), key file path (`infra/aws/tazama-aws.pem`), and remote paths are all pre-configured to match this deployment. Instance IDs and IPs are read dynamically from `tofu output` at runtime so they never need to be hardcoded.

Prerequisite: `tofu apply` must have been run (step C.12). The scripts SSH to instances whose IDs come from the live OpenTofu state.

The scripts talk to EC2 instances through the EICE ProxyCommand - the same mechanism as manual SSH (see Phase E.1). No port 22 needs to be open anywhere; authentication is entirely via the IAM profile on your local AWS CLI session.

To deploy everything in one go:

```
cd full-stack-docker-tazama\infra\aws\scripts
.\deploy.ps1 -Password 'your-strong-password'
```

The `-Password` value is applied in-memory to all PostgreSQL and Keycloak admin credential variables on Server A and Server B. It is never written to a committed file.

Or run the three deploy scripts individually if you want to validate each server before moving on to the next. The runnable scripts are, in order:

```
.\deploy-core.ps1          -Password 'your-strong-password' # Server A: waits for bootstrap, starts tazama-
core
.\deploy-extensions.ps1    -Password 'your-strong-password' # Server A: adds DEMS/DEAPI; Server B: starts
tazama-extensions
.\deploy-biar.ps1          # Server C: waits for bootstrap, starts
tazama-biar
```

Note: `helpers.ps1` (documented in D.1) is a function library, not a script to run directly. It is dot-sourced automatically by each of the three scripts above. Running it directly loads its functions silently and exits -- that is the expected behaviour.

D.1 helpers.ps1

infra/aws/scripts/helpers.ps1

- shared functions. Dot-sourced by every other script.

Function	Purpose
Get-TofuOutputs	Runs <code>tofu output -json</code> and returns a hashtable of instance IDs and private IPs
Invoke-RemoteCommand	SSH to an EC2 instance via EICE ProxyCommand and run a shell command
Copy-ToRemote	SCP a file to an EC2 instance via EICE
Set-RemoteEnvOverlay	Reads a KEY=VALUE overlay file and applies each entry to a remote <code>.env</code> using <code>sed</code>
Wait-Bootstrap	Polls every 30 s until <code>/home/ec2-user/.bootstrap-complete</code> exists on the remote instance

EICE SSH ProxyCommand used internally:

```
ProxyCommand aws ec2-instance-connect open-tunnel --instance-id %h --remote-port %p --region ap-south-1 --profile tazama
```

The SSH hostname is the EC2 instance ID (e.g. `i-0abc123`). The `open-tunnel` command communicates over stdio - OpenSSH on Windows supports this via `ProxyCommand` in a per-connection temp SSH config file. No TCP port 22 is exposed anywhere.

D.2 deploy-core.ps1

infra/aws/scripts/deploy-core.ps1

1. Reads `server_a_instance_id` from `tofu output`
2. Calls `Wait-Bootstrap` (polls up to 15 min for `.bootstrap-complete`)
3. Pulls the latest repo changes on Server A (`git pull`)
4. Copies `core/.env` and the Keycloak realm JSON to Server A
5. Starts the tazama-core stack on Server A with the full compose chain, retrying up to 3× if Postgres is still initialising (see note below)

Parameters:

Parameter	Description
-NoPull	Skip <code>--pull always</code> on the <code>docker compose up</code> command. Use when images are already present on the host (e.g. retrying after a failed start). The first retry attempt within the script automatically drops the pull flag regardless.
-Password	PostgreSQL superuser password and Keycloak admin password. Applied in-memory via <code>Set-RemoteEnvOverlay</code> to <code>core/.env</code> and all <code>core/env/</code> service env files on Server A. Sets <code>POSTGRES_PASSWORD</code> , <code>RAW_HISTORY_DATABASE_PASSWORD</code> , <code>CONFIGURATION_DATABASE_PASSWORD</code> , <code>EVENT_HISTORY_DATABASE_PASSWORD</code> , <code>EVALUATION_DATABASE_PASSWORD</code> , <code>NEXT_PUBLIC_PG_PASSWORD</code> , and <code>KEYCLOAK_ADMIN_PASSWORD</code> . If omitted, local-dev defaults (<code>unused / password</code>) are left in place -- do not omit for any non-development deployment.

Full compose chain on Server A:

```
docker compose -p tazama-core \  
-f ./docker-compose.base.infrastructure.yaml \  
-f ./docker-compose.base.override.yaml      <- always position 2; publishes NATS/PG/Valkey \  
-f ./docker-compose.full.cfg.yaml \  
-f ./docker-compose.hub.core.yaml \  
-f ./docker-compose.full.rules.yaml \  
-f ./docker-compose.base.auth.yaml \  
-f ./docker-compose.hub.relay.yaml \  
-f ./docker-compose.hub.logs.base.yaml \  
-f ./docker-compose.utils.pgadmin.yaml
```



```
-f ./docker-compose.utils.hasura.yaml \
up -d [--pull always]
```

Hasura / Postgres race condition: Hasura has a `depends_on: postgres: condition: service_healthy`. On a cold first boot, Postgres runs schema migration scripts before reporting healthy — this can take longer than the default healthcheck grace period. The script automatically retries `docker compose up` up to 3 times with a 30-second gap. The first attempt uses `--pull always`; subsequent retries skip the pull (images are already present). If the stack still fails after all 3 attempts, run `.\deploy-core.ps1 -NoPull` to retry without re-pulling images.

D.3 `deploy-extensions.ps1`

[infra/aws/scripts/deploy-extensions.ps1](#)

1. **Server A** - adds DEMS + DEAPI to the running `tazama-core` project:

```
cd extensions/ && docker compose -p tazama-core \
-f ./docker-compose.dev.extensions.apis.yaml up -d
```

These APIs must be reachable before TCS/TRS backends start on Server B.

2. **Server B** - waits for bootstrap (up to 15 min)
3. **Server B** - applies `templates/env-extensions.tpl` overlay to `extensions/.env`: sets `SERVER_A_HOST=core.tazama.internal` and `SERVER_B_HOST=extensions.tazama.internal`
4. **Server B** - if `-Password` is supplied, applies a second in-memory credential overlay to `extensions/.env` and all `extensions/env/` service env files: sets `POSTGRES_PASSWORD`, `DB_PASSWORD`, `SPRING_DATASOURCE_PASSWORD`, and `CONFIGURATION_DATABASE_PASSWORD`
5. **Server B** - SCP `core/auth/test-public-key.pem` → `extensions/auth/` (required by TCS/TRS for JWT validation; on single-machine dev the bat script copies it automatically - here we do it from the local repo)
6. **Server B** - starts the extensions stack:

```
docker compose -p tazama-extensions \
-f ./docker-compose.extensions.infrastructure.yaml \
-f ./docker-compose.hub.extensions.yaml \
up -d
```

TCS (backend + frontend), TRS (backend + frontend), and CMS frontend pull from DockerHub. CMS backend still builds from source until `tazamaorg/case-management-system-backend` is published.

Parameters:

Parameter	Description
<code>-NoPull</code>	Skip <code>--pull always</code> on <code>docker compose up</code> .
<code>-Password</code>	PostgreSQL password for Server B's database and all extension service clients. Applied in-memory -- never written to a committed file. If omitted, local-dev defaults (<code>unused</code>) are left in place.

D.4 `deploy-biar.ps1`

[infra/aws/scripts/deploy-biar.ps1](#)

1. Waits for bootstrap on Server C (up to 15 min)
2. Pulls the latest repo on Server C — ensures the server is on the correct branch
3. Copies `biar/.env` to Server C (gitignored — never committed)
4. Applies `templates/env-biar.tpl` overlay to `biar/.env` — sets all three host vars:

- `SERVER_A_HOST=core.tazama.internal`
 - `SERVER_B_HOST=extensions.tazama.internal`
 - `SERVER_C_HOST=biar.tazama.internal`
5. Creates the Tazama warehouse directory: `sudo mkdir -p /opt/Tazama_Warehouse` — bind-mounted by automation-orchestrator and datalakehouse-api
6. **Staged Ozone startup** (SCM must fully initialise before OM and datanodes):
- Starts `scm` only, waits 20 s for SCM to initialise
 - Starts `om`, waits 15 s for OM to register
 - Brings up the full stack:

```
docker compose -p tazama-biar \
  -f ./docker-compose.biar.infrastructure.yaml \
  -f ./docker-compose.hub.biar.yaml \
  -f ./docker-compose.utils.init.yaml \
  up -d [--pull always]
```

The `aws-cli` init container creates the `tazama` Ozone bucket automatically once S3G is healthy. The `nifi-init` container polls the NiFi API and injects the parameter context + template when NiFi becomes ready (up to 5 minutes).

Parameters:

Parameter	Description
<code>-NoPull</code>	Skip <code>--pull always</code> on <code>docker compose up</code> . Use when images are already present on Server C.

Server A and Server B must be up before this script is run — NiFi connects to PostgreSQL (:15432) on Server A and PostgreSQL (:15433) on Server B. These connections are made when NiFi flows start, not at container startup, so the containers will start but flows will fail until both database servers are reachable.

D.5 `deploy-lakehouse.ps1`

[infra/aws/scripts/deploy-lakehouse.ps1](#)

Stages the Tazama Lakehouse archive through S3 and unpacks it on Server C into `/opt/Tazama_Warehouse`. Run this **after** `deploy-biar.ps1` has completed — the target directory is created by that script and the automation-orchestrator and datalakehouse-api containers must be up before any workflows access the warehouse.

The Lakehouse archive (`Tazama_Lakehouse.zip`) is not committed to the repository. Obtain it from the Tazama dev team and place it anywhere accessible from your local machine.

Why S3 and not SCP? The archive is typically 3-4 GB. EICE tunnels are stdio-based and throttled — SCP over EICE at that size would take hours or time out. Uploading to S3 from your workstation and then pulling it down on Server C (same AWS region, internal network) is dramatically faster and more reliable.

```
cd full-stack-docker-tazama\infra\aws
.\scripts\deploy-lakehouse.ps1 -ZipPath "D:\DevTools\Tazama\Tazama_Lakehouse.zip"
```

What the script does:

1. Reads `server_c_instance_id` and `state_bucket` from `tofu output`
2. Configures the AWS CLI S3 multipart chunk size to 256 MB (reduces upload to ~15 parts for a 3-4 GB file, avoiding connection-drop failures at the default 8 MB / ~475 parts)
3. Uploads the zip to `s3://<state_bucket>/lakehouse-staging/Tazama_Lakehouse.zip` from your local machine
4. On Server C: downloads the zip from S3 using the instance IAM role (no credentials needed)
5. Installs `unzip` on Server C if not already present
6. Runs `sudo unzip -o` to `/` — the zip already contains the full path (`opt/Tazama_Warehouse/...`) so the files land at `/opt/Tazama_Warehouse/` directly. Do **not** pass `-d /opt/Tazama_Warehouse` or the path will double-nest.
7. Deletes the zip from Server C and removes the staging object from S3

The instance IAM role has a scoped read policy on the `lakehouse-staging/` prefix of the state bucket — added to `main.tf` as `aws_iam_role_policy.s3_staging_read`. Your local AWS profile requires `s3:PutObject` on the same bucket (already granted when you created the bucket in Phase B).

Parameters:

Parameter	Description
<code>-ZipPath</code>	(Required) Local path to <code>Tazama_Lakehouse.zip</code>

Re-deployment note: The script is idempotent — re-running with `-ZipPath` will overwrite existing warehouse files (`unzip -o`). Existing files not present in the zip are left in place.

Upload keeps failing? If the upload fails repeatedly mid-transfer, run these once before retrying:

```
aws configure set s3.multipart_chunksize 256MB --profile tazama
aws configure set s3.multipart_threshold 256MB --profile tazama
```

The script sets these automatically, but an interrupted previous run may have left a partial multipart upload in progress. Check with:

```
aws s3api list-multipart-uploads --bucket <state_bucket> --profile tazama --region ap-south-1
```

Abort any stale uploads before retrying.

D.7 `deploy.ps1`

[infra/aws/scripts/deploy.ps1](#)

Orchestrator - calls `deploy-core.ps1`, `deploy-extensions.ps1`, and `deploy-biar.ps1` in sequence. Safe to run immediately after `tofu apply`; each sub-script waits for its server's bootstrap internally.

Parameters:

Parameter	Description
<code>-</code>	Passed through to <code>deploy-core.ps1</code> and <code>deploy-extensions.ps1</code> . Sets PostgreSQL and Keycloak admin passwords on both servers.
<code>-NoPull</code>	Passed through to all three sub-scripts. Skips <code>--pull</code> always on <code>docker compose up</code> .

```
cd full-stack-docker-tazama\infra\aws\scripts
.\deploy.ps1 -Password 'your-strong-password'
```

D.8 `teardown.ps1`

[infra/aws/scripts/teardown.ps1](#)

Stops all Docker Compose stacks in reverse order (C → B → A). Volumes are preserved by default.

```
.\teardown.ps1 # stop containers; keep data
.\teardown.ps1 -RemoveVolumes # stop containers AND delete all data
```

`-RemoveVolumes` requires typing `YES` to confirm. Use it before `tofu destroy` to avoid orphaned volumes consuming disk on the (soon to be deleted) instances.

To destroy all AWS infrastructure after teardown:

```
cd full-stack-docker-tazama\infra\aws
tofu destroy
```

Phase E: Sandbox Access

After Phase D completes, the services are running and reachable from within the VPC. The private subnet means there is no public IP on any server — you must choose how to expose the sandbox before it can be accessed.

	Option 1: SSH Tunnelling	Option 2: ALB	Option 3: Custom Domain
Access URL	http://localhost:<port>	http://<alb-dns-name>:<port>	https://<service>.<your-zone>
Who can use it	Anyone with your AWS credentials locally	Anyone with a network connection	Anyone with a network connection
Access cost	None	~\$20-30/month (ALB + LCUs)	~\$20-30/month + domain costs
Setup	Run a script locally	One <code>tofu apply</code>	ALB + NS delegation + <code>tofu apply</code>
Use for	Developer/operator access, CI/CD with AWS creds	Sharing with teammates, demos, clients	Persistent shared environments, HTTPS required

All options use the same port numbers as the local Docker deployment. The options are not mutually exclusive — you can run tunnels alongside an active ALB. Option 3 requires Option 2 (the ALB) and adds a clean HTTPS subdomain layer on top.

E.1 Option 1: SSH Tunnelling (private sandbox)

Each tunnel script opens a blocking SSH connection through the EICE endpoint and forwards local ports to the matching container ports on the remote server. No port 22, no public IPs — authentication is via your local AWS IAM session.

There are two ways to run the tunnels:

All three servers at once (recommended)

`tunnel-all.ps1` launches all three SSH connections as background jobs and waits on them in a single terminal window. Ctrl+C stops all three cleanly.

```
cd full-stack-docker-tazama\infra\aws\scripts
.\tunnel-all.ps1
```

Per-server (targeted access)

Each script is blocking — it holds the terminal while the tunnel is open. Run each in a **separate** terminal window if you want all the tunnels opened, or just run the one for the server you want to access:

```
# Terminal 1 – Server A: TMS, Admin, Auth, Keycloak, Hasura, pgAdmin, NATS, PostgreSQL, DEAPI, DEMS
.\tunnel-server-a.ps1

# Terminal 2 – Server B: TCS, TRS, CMS frontends + backends, OpenSearch, pgAdmin (ext)
.\tunnel-server-b.ps1

# Terminal 3 – Server C: NiFi, Solr, Ozone
.\tunnel-server-c.ps1
```

Once tunnels are open, every service is reachable at `http://localhost:<port>`. Your existing Postman environment (with `localhost:*` base URLs) works without modification. Proceed to Phase F to validate.

Port map — Server A (`tunnel-server-a.ps1`):

Local port	Service
5000	TMS API
5100	Admin API
3020	Auth Service
8080	Keycloak
6100	Hasura
5050	pgAdmin (core DB)
14222	NATS
15432	PostgreSQL (core)
3001	DEAPI
3002	DEMS

Port map — Server B (`tunnel-server-b.ps1`):

Local port	Service
3010	TCS Backend
5173	TCS Frontend
3005	TRS Backend
5174	TRS Frontend
3090	CMS Backend
5175	CMS Frontend
9200	OpenSearch
15433	PostgreSQL (extensions)
5051	pgAdmin (extensions DB)

Port map — Server C (`tunnel-server-c.ps1`):

Local port	Service
7619	Automation Orchestrator API
8000	JupyterHub
8088	NiFi UI
8282	Datalakehouse API
8983	Solr admin
9876	Ozone SCM
9878	Ozone S3G
9888	Ozone Recon UI

E.2 Option 2: ALB — public sandbox

The ALB module (`modules/alb/`) is already in the Terraform codebase but is not deployed by default. Enable it with a single variable and apply:

E.2.1 Enable the ALB

Recommended: include `alb.tfvars` in the initial Phase C plan/apply (see C.11) so that `deploy-core.ps1` can inject the ALB hostname into Keycloak on first boot. If you are following this section after Phase D has already run, see the Keycloak restart note below.

`alb.tfvars` is already in the repo with `enable_alb = true` — no editing required. Pass it as a second var-file:

```
cd full-stack-docker-tazama\infra\aws
tofu apply -var-file terraform.tfvars -var-file alb.tfvars
```

OpenTofu plans and applies only the ALB delta. The EC2 instances and all running containers are untouched. Expected additions: roughly +30 resources (1 ALB, 14 target groups, 14 port-based HTTP listeners, security group rules).

[!WARNING] Once `domain.tfvars` has been applied (Phase E.3), always include it in every subsequent `tofu apply`. Omitting it tells OpenTofu to remove `module.dns_public`, which destroys the Route 53 public zone and all its records. If that has already happened, use the full three-file command for all future applies:

```
tofu apply -var-file terraform.tfvars -var-file alb.tfvars -var-file domain.tfvars
```

Always run `tofu plan` first and verify `0 to destroy` before applying.

Already applied without `alb.tfvars`? No teardown needed. Just add `-var-file alb.tfvars` and re-run `tofu apply` — exactly the same command.

Applied ALB after Phase D? Keycloak is already running with `KC_HOSTNAME=localhost`. Fix it by re-running `deploy-core.ps1 -NoPull` — the script will detect the ALB DNS name, update `core/.env`, and the compose up will recreate the Keycloak container with the correct hostname. Alternatively, SSH to Server A and restart just Keycloak:

```
cd ~/full-stack-docker-tazama/core
echo "KEYCLOAK_HOSTNAME=<alb-dns-name>" >> .env
docker compose -p tazama-core restart keycloak
```

E.2.2 Get the ALB DNS name

```
tofu output alb_dns_name
```

Every service is now reachable at `http://<alb-dns-name>:<port>`, e.g.:

```
http://tazama-alb-1234567890.ap-south-1.elb.amazonaws.com:5000 ← TMS
http://tazama-alb-1234567890.ap-south-1.elb.amazonaws.com:5173 ← TCS UI
```

Update your Postman environment: set the base URL variable to `http://<alb-dns-name>` (no port — port goes in each request URL).

E.2.3 What the ALB module creates

Health check paths:

Service	Path	Status matcher
Keycloak	/health/ready	200-399
Hasura	/healthz	200-399
NiFi	/nifi-api/system-diagnostics	200-399
JupyterHub	/hub/health	200-399

Service	Path	Status matcher
All others	/health	200-399

Key outputs (used by Phase G custom domain upgrade):

- `alb_dns_name` — DNS name for port-based access
- `alb_zone_id` — ALB Route 53 zone ID, for Phase G alias records
- `alb_arn` — ALB ARN, for Phase G HTTPS listener
- `target_group_arns` — map of service name → TG ARN, for Phase G host-based routing

Files:

- `infra/aws/modules/alb/variables.tf`
- `infra/aws/modules/alb/main.tf`
- `infra/aws/modules/alb/outputs.tf`

E.2.4 Rollback

To remove the ALB, simply omit `alb.tfvars`:

```
tofu apply -var-file terraform.tfvars
```

SSH tunnel access is unaffected by the ALB being present or absent.

E.3 Option 3: Custom Domain + HTTPS

Option 3 builds on top of Option 2 (the ALB must be active first). It replaces the port-based ALB URLs with clean HTTPS subdomains:

```
http://tazama-alb-1234567890.ap-south-1.elb.amazonaws.com:5000 ← before
https://tms.<your-zone>                                       ← after
```

When Option 3 is active, the port-based HTTP listeners from Option 2 remain in place so EICE tunnels and existing Postman environments continue to work.

E.3.1 Prerequisites

#	Requirement
1	Your domain registered and accessible at your registrar
2	Option 2 (ALB) active — <code>alb.tfvars</code> applied
3	NS delegation for <code><your-zone></code> added at your registrar (see E.3.4)

E.3.2 Enable Option 3

Copy `domain.tfvars.example` to `domain.tfvars` (gitignored) and set your zone name:

```
cd full-stack-docker-tazama\infra\aws
Copy-Item domain.tfvars.example domain.tfvars
# Edit domain.tfvars: replace "env.your-domain.com" with your actual zone
notepad domain.tfvars
```

Then apply, stacking all three var-files:

```
tofu apply -var-file terraform.tfvars -var-file alb.tfvars -var-file domain.tfvars
```

OpenTofu will:

1. Create a Route 53 public hosted zone for `<your-zone>` (e.g. `env.your-domain.com`)
2. Request an ACM wildcard certificate `*.<your-zone>`
3. Write the DNS validation CNAME record into the hosted zone
4. Wait for ACM to validate the certificate (requires NS delegation to be in place — see E.3.4)
5. Create Route 53 alias records: `<service>.<your-zone> → ALB`
6. Add an HTTPS:443 listener on the ALB with host-based routing rules

Expected plan summary:

```
Plan: 34 to add, 0 to change, 0 to destroy.
```

Changes to Outputs:

```
~ acm_certificate_arn      = "" -> (known after apply)
~ public_zone_id          = "" -> (known after apply)
~ public_zone_nameservers = [] -> (known after apply)
```

The outputs were empty strings while Option 3 was inactive — they will now be populated with real values. OpenTofu will prompt for confirmation before proceeding:

```
Do you want to perform these actions?
  OpenTofu will perform the actions described above.
  Only 'yes' will be accepted to approve.

Enter a value: yes
```

Partial apply note: If a previous apply failed mid-way (e.g. due to a missing ACM permission — see Troubleshooting), OpenTofu will have already created some resources. On retry, the plan will show fewer additions. This is expected — OpenTofu skips what already exists and only creates what remains. For example, if the Route 53 zone and service records were created but the ACM certificate was not, the retry plan will show:

```
Plan: 19 to add, 0 to change, 0 to destroy.
```

Changes to Outputs:

```
~ acm_certificate_arn = "" -> (known after apply)
```

Type **yes** to proceed.

Note: If NS delegation is **not** yet in place, step 4 will time out after ~10 minutes. Omit `-var-file domain.tfvars` and re-apply to destroy the partial resources, get the NS records added at your registrar, then re-apply with all three var-files.

E.3.3 Get the Nameserver values (after first apply)

Run this immediately after `tofu apply` succeeds:

```
tofu output public_zone_nameservers
```

Sample output:

```
[
  "ns-1234.awsdns-12.org.",

```



```
"ns-567.awsdns-45.co.uk.",
"ns-89.awsdns-01.com.",
"ns-2345.awsdns-67.net.",
]
```

These are the four values your registrar admin needs.

[!WARNING] **NS values change if the zone is recreated.** Every time `tofu destroy` removes and `tofu apply` recreates the Route 53 zone, AWS assigns a new set of nameservers. If you have previously sent NS values to your registrar and then destroyed/recreated the zone (e.g. during troubleshooting), the values on file at the registrar will be stale and DNS will not resolve. Always retrieve fresh NS values after any apply that recreates the zone:

```
aws route53 list-hosted-zones-by-name --dns-name "<your-zone>" --profile tazama `
  --query "HostedZones[0].Id" --output text | ForEach-Object {
    aws route53 get-hosted-zone --id $_ --profile tazama `
      --query "DelegationSet.NameServers" --output table
  }
```

Send the updated values to your registrar and ask them to replace the existing NS records. Until the registrar has the correct current values, ACM validation will time out and `tofu apply` will block indefinitely at `aws_acm_certificate_validation`.

E.3.4 Registrar request — NS delegation

Ask your registrar or DNS admin to add 4 NS records to the parent zone (e.g. `your-domain.com`) so that `<your-zone>` resolves via your AWS Route 53 hosted zone.

Sample ticket/email subject: `<your-zone>` subdomain NS delegation, the part before the `<your-domain>`

Name	Type	Value (TTL 300)
<code><your-zone></code>	NS	<code><paste ns-1 from tofu output></code>
<code><your-zone></code>	NS	<code><paste ns-2 from tofu output></code>
<code><your-zone></code>	NS	<code><paste ns-3 from tofu output></code>
<code><your-zone></code>	NS	<code><paste ns-4 from tofu output></code>

E.3.5 Verify HTTPS is live

After your registrar confirms the NS records are in place, DNS propagation typically takes 1-5 minutes. Then run:

```
# Set your zone once – replace with your domain_zone value
$zone = "env.your-domain.com"

# Check TMS (or your equivalent entry point)
Invoke-RestMethod "https://tms.$zone/health"

# Check Keycloak
Invoke-RestMethod "https://keycloak.$zone/health/ready"

# Check NiFi
Invoke-RestMethod "https://nifi.$zone/nifi-api/system-diagnostics"
```

All should return `200 OK`.

E.3.6 Update application configs for HTTPS

After HTTPS is live, update the following before redeploying extensions:

Keycloak — update frontend URL via Admin UI or env override:

```
KC_HOSTNAME_URL=https://keycloak.<your-zone>
KC_HOSTNAME_ADMIN_URL=https://keycloak.<your-zone>
```

TCS / TRS / CMS — update the API base URL in each service's `.env` overlay (stored in `infra/aws/config/extensions.env`):

```
TCS_API_BASE_URL=https://tcs-api.<your-zone>
TRS_API_BASE_URL=https://trs-api.<your-zone>
CMS_API_BASE_URL=https://cms-api.<your-zone>
AUTH_BASE_URL=https://auth.<your-zone>
```

Then redeploy extensions:

```
.\scripts\deploy-extensions.ps1
```

E.3.7 Subdomain map

Subdomain	Service	Target
tms.<your-zone>	Transaction Monitoring Service	Server A :5000
admin.<your-zone>	Admin API	Server A :5100
auth.<your-zone>	Auth Service	Server A :3020
keycloak.<your-zone>	Keycloak	Server A :8080
pgadmin.<your-zone>	pgAdmin (core DB)	Server A :5050
hasura.<your-zone>	Hasura	Server A :6100
tcs.<your-zone>	TCS Frontend	Server B :5173
tcs-api.<your-zone>	TCS Backend API	Server B :3010
trs.<your-zone>	TRS Frontend	Server B :5174
trs-api.<your-zone>	TRS Backend API	Server B :3005
cms.<your-zone>	CMS Frontend	Server B :5175
cms-api.<your-zone>	CMS Backend API	Server B :3090
pgadmin-ext.<your-zone>	pgAdmin (extensions DB)	Server B :5051
nifi.<your-zone>	NiFi UI	Server C :8088
jupyter.<your-zone>	JupyterHub (multi-user analytics)	Server C :8000
datalakehouse-api.<your-zone>	Datalakehouse API	Server C :8282
automation-orchestrator.<your-zone>	Automation Orchestrator API	Server C :7619

E.3.8 Rollback

To remove the custom domain, omit `domain.tfvars`:

```
tofu apply -var-file terraform.tfvars -var-file alb.tfvars
```

Port-based HTTP listeners on the ALB are unaffected.

[!WARNING] **Omitting `domain.tfvars` destroys the Route 53 zone.** OpenTofu treats the absence of a var-file as "remove everything that var-file enabled". This means the Route 53 public zone — and all its records — will be destroyed. If you later re-

enable Option 3, AWS will create a **new** zone with **different** nameservers, and NS delegation will need to be repeated at your registrar.

Only omit `domain.tfvars` if you genuinely want to remove the custom domain. For all other applies (server rebuilds, ALB changes, config updates) always include it:

```
tofu apply -var-file terraform.tfvars -var-file alb.tfvars -var-file domain.tfvars
```

Phase F: Validation

All three servers are up with containers running.

F.3 Server A smoke test

Pre-requisite: `tunnel-server-a.ps1` running in a separate terminal (see Phase E.1).

```
cd full-stack-docker-tazama\infra\aws\scripts
.\tunnel-server-a.ps1
```

Postman environment: Import `postman/environments/Tazama-Docker-Compose.postman_environment.json`. All `localhost:*` values already match the tunnel script port map. No changes needed.

Postman collection: `postman/3.1. (NO-AUTH) Public DockerHub Full-Service Test.postman_collection.json`

This is the correct collection for the `full.cfg` + `full.rules` deployment (all rule processors, single generic typology).

Run the full collection. The collection:

1. Generates a random pacs.008 + pacs.002 transaction pair
2. Submits to TMS API on `:5000`
3. Polls the result databases (via Hasura on `:6100`) to verify the transaction flowed through all processors and a result was committed

Expected outcome: All requests green, result record present in the `evaluation` database.

Quick health check (without Postman):

```
# TMS API - should return {"status":"ok"} or similar
Invoke-RestMethod http://localhost:5000/health

# Admin API
Invoke-RestMethod http://localhost:5100/health

# Auth Service
Invoke-RestMethod http://localhost:3020/health
```

F.4 DEMS / DEAPI verification on Server A

DEMS and DEAPI are added to the `tazama-core` project by `deploy-extensions.ps1` (step 3). Verify both containers are running with the tunnel open:

```
# DEAPI (Data Enrichment API)
Invoke-RestMethod http://localhost:3001/health

# DEMS (Data Enrichment Monitoring Service)
Invoke-RestMethod http://localhost:3002/health
```

If either is absent, check that `deploy-extensions.ps1` step 3 completed without error. The most common cause is `GH_TOKEN` not being available — re-run the script; the `bash -l` wrapper ensures the token is loaded from `/etc/environment`.

F.5 Server B validation

Pre-requisite: `deploy-extensions.ps1` has completed successfully.

Open a second tunnel (keep Server A tunnel running — TCS/TRS/CMS need to reach Server A):

```
.\tunnel-server-b.ps1
```

Health checks:

```
# TCS (Connection Studio) backend
Invoke-RestMethod http://localhost:3010/health

# TRS (Rule Studio) backend
Invoke-RestMethod http://localhost:3005/health

# CMS (Case Management) backend
Invoke-RestMethod http://localhost:3090/health

# OpenSearch
Invoke-RestMethod http://localhost:9200/_cluster/health | ConvertTo-Json
```

Browser checks:

- TCS frontend: `http://localhost:5173`
- TRS frontend: `http://localhost:5174`
- CMS frontend: `http://localhost:5175`

Cross-server connectivity (Server A): TCS/TRS backends contact Server A for JWT validation (port 3020) and NATS relay (port 14222). If the frontend loads but API calls fail with 401/CORS, the `SERVER_A_HOST` overlay in `extensions/.env` was not applied — check the `env-extensions.tpl` template and re-run step 6 of `deploy-extensions.ps1` manually.

Cross-server connectivity (Server C — datalakehouse-api): The CMS backend calls the `datalakehouse-api` on Server C directly (not via the ALB). Verify reachability from Server B:

```
cd full-stack-docker-tazama\infra\aws
. .\scripts\helpers.ps1
$out = Get-ToFuOutputs
Invoke-RemoteCommand -InstanceId $out.ServerB_InstanceID -Command 'curl -s -o /dev/null -w "%{http_code} %
{time_total}s" --max-time 10 http://biar.tazama.internal:8282/health'
# Expected: 200 <time>s
```

If this returns `000` (connection failure), check two things:

1. Server B's security group — it must be `tazama-server-b-sg`, **not** `tazama-server-a-sg`:

```
aws ec2 describe-instances --profile tazama --region ap-south-1 `
  --instance-ids $out.ServerB_InstanceID `
  --query "Reservations[0].Instances[0].SecurityGroups[*].GroupName" --output text
```

If it shows `tazama-server-a-sg`, correct it: `aws ec2 modify-instance-attribute --profile tazama --region ap-south-1 --instance-id $out.ServerB_InstanceID --groups <server-b-sg-id>`

2. Server C's SG must have an inbound rule for TCP 8282 from `tazama-server-b-sg`. A `tofu plan` will reveal and a `tofu apply` will correct any such drift.

F.6 Server C validation

Pre-requisite: `deploy-biar.ps1` has completed successfully.

```
.\tunnel-server-c.ps1
```

Browser/API checks:

- NiFi UI: <http://localhost:8088/nifi> — login with `admin` / `admin123456789`
- Solr admin: <http://localhost:8983/solr>
- Ozone Recon: <http://localhost:9888>
- JupyterHub: <http://localhost:8000> — see first-time setup note below

```
# Automation Orchestrator API (FastAPI)
Invoke-RestMethod http://localhost:7619/health

# Datalakehouse API (FastAPI)
Invoke-RestMethod http://localhost:8282/health

# JupyterHub
Invoke-RestMethod http://localhost:8000/hub/health
```

JupyterHub first-time setup: Signup is disabled by default (`open_signup = False`). The first admin account must be created and then authorized manually:

1. Go to <http://localhost:8000/hub/signup> (or <https://jupyter.<your-zone>/hub/signup>)
2. Sign up using the `JUPYTERHUB_ADMIN` username (default: `admin`, overridden by the `JUPYTERHUB_ADMIN` env var in `jupyterlab.env`)
3. Go to <http://localhost:8000/hub/authorize> and click **Authorize** next to the admin account
4. Log in — the admin account is now active

Subsequent users: Direct them to </hub/signup>. Their accounts will appear at </hub/authorize> for admin approval. Once approved they can log in and will appear in the admin panel at </hub/admin>.

Forgot admin password? Reset it directly in the SQLite database:

```
Invoke-RemoteCommand -InstanceId $out.ServerC_InstanceId -Command @"
docker exec biar-jupyterhub python3 -c "
import sqlite3, bcrypt
new_password = b'NewPassword123!'
hashed = bcrypt.hashpw(new_password, bcrypt.gensalt(12))
conn = sqlite3.connect('/data/jupyterhub.sqlite')
c = conn.cursor()
c.execute("UPDATE users_info SET is_authorized=1, password=? WHERE username='admin'", (hashed,))
conn.commit()
conn.close()
print('Done.')
"@
```

Then log in with the new password.

NiFi → Server A PostgreSQL connectivity: NiFi connects to PostgreSQL on Server A (`:15432`) at startup. In the NiFi UI, check the Controller Services tab. Any DBCPConnectionPool service that targets `core.tazama.internal:15432` should show **Enabled** status. A **Disabled** or **Invalid** service indicates the `SERVER_A_HOST` overlay was not applied — check `env-biar.tpl` and re-run the overlay step manually:

```
cd full-stack-docker-tazama\infra\aws\scripts
. .\helpers.ps1
```

```
$out = Get-TofuOutputs
$overlayFile = Join-Path $PSScriptRoot '..\templates\env-biar.tpl'
$remoteEnvFile = '/home/ec2-user/full-stack-docker-tazama/biar/.env'
Set-RemoteEnvOverlay -InstanceId $out.ServerC_InstanceID -OverlayFile $overlayFile -RemoteEnvFile
$remoteEnvFile
# then restart the biar stack
Invoke-RemoteCommand -InstanceId $out.ServerC_InstanceID -Command "cd /home/ec2-user/full-stack-docker-
tazama/biar && docker compose -p tazama-biar restart"
```

F.7 Full teardown

```
cd full-stack-docker-tazama\infra\aws\scripts
.\teardown.ps1 # stop containers; volumes preserved
.\teardown.ps1 -RemoveVolumes # stop containers AND delete all volumes (type YES to confirm)
```

To also destroy all AWS infrastructure:

```
cd full-stack-docker-tazama\infra\aws
tofu destroy
```

F.8 Redeploy without `tofu destroy` (containers only)

Used when the EC2 instances are still running but containers were torn down via `teardown.ps1`. The repo and `.env` files are already on each server from the previous deploy.

```
cd full-stack-docker-tazama\infra\aws\scripts
.\deploy-core.ps1
.\deploy-extensions.ps1
.\deploy-biar.ps1
```

The scripts are idempotent — `docker compose up -d` is a no-op for containers already running.

Images already present? If the previous teardown left Docker images on the hosts (i.e. volumes were removed but not images), you can skip the re-pull to speed up the core stack start:

```
.\deploy-core.ps1 -NoPull
```

`deploy-extensions.ps1` and `deploy-biar.ps1` do not have a `-NoPull` switch — they always pull.

F.9 Redeploy from scratch (after `tofu destroy`)

After `tofu destroy`, all EC2 instances, the EICE endpoint, and all associated networking are gone. The S3 state bucket and DynamoDB lock table survive — they are the only resources not managed by the `tofu destroy` scope.

If Option 3 (custom domain) was active: `tofu destroy` also destroys the Route 53 public zone. AWS will assign new nameservers when the zone is recreated — NS delegation at your registrar will need to be updated. See E.3.3 for how to retrieve the new nameserver values after apply.

If you are only rebuilding containers (F.8 — no `tofu destroy`), the Route 53 zone is untouched and NS delegation does not need to be repeated. The zone and its nameservers are infrastructure-layer resources that survive container-level redeployments.

```
cd full-stack-docker-tazama\infra\aws
```

```
# Review what will be created
# Include all active var-files - omitting domain.tfvars here would destroy the Route 53 zone
tofu plan -var-file terraform.tfvars -var-file alb.tfvars -var-file domain.tfvars -out tfplan

# Apply (will re-provision all three instances and networking)
tofu apply tfplan

# Wait for bootstraps, then deploy in order
cd scripts
.\deploy-core.ps1
.\deploy-extensions.ps1
.\deploy-biar.ps1
```

Note: New instances clone the branch specified by `var.repo_branch` (default `dev`) via `bootstrap.sh.tpl`. The branch-switch step in the deploy scripts (`deploy-core.ps1`, `deploy-extensions.ps1`, `deploy-biar.ps1`) reads `$Script:RepoBranch` from `helpers.ps1` (also defaults to `dev`) and is a no-op for freshly bootstrapped instances but acts as a safety net if a server was provisioned against an older bootstrap.

To switch the target branch across **all** scripts and the bootstrap template, change two values:

1. `$Script:RepoBranch` in `infra/aws/scripts/helpers.ps1`
2. `repo_branch` default (or `terraform.tfvars` override) in `infra/aws/variables.tf`

Phase G: Security Hardening

This section describes the default security posture of the beta deployment and the concrete steps required to raise it to a level appropriate for use beyond an isolated developer sandbox.

G.1 - Default security posture

The deployment as provisioned out of the box has the following characteristics:

Network boundary (good)

- All services run inside a private VPC subnet (`10.0.1.0/24`) with no inbound internet route
- Access to EC2 instances requires AWS IAM credentials and EICE — there is no open port 22
- ALB-exposed services (`nifi`, `jupyter`, `tms`, `keycloak`, etc.) sit behind HTTPS with ACM-managed certificates and Keycloak-enforced authentication at the listener rule level

Credentials (weak — requires action before beta use)

- All three stacks (`core/`, `extensions/`, `biar/`) were originally committed to a public GitHub repository with plaintext default passwords (`unused`, `password`, `admin123456789`, `tazama`)
- The `-Password` deploy-script parameter covers PostgreSQL and Keycloak on Servers A and B. All other service credentials (Redis, CouchDB, TRS signing key, Hasura, NiFi, Ozone, OpenSearch, SFTP, pgAdmin, OAuth client secret, relay auth, CMS Flowable) remain at their committed defaults unless changed manually
- On the current beta deployment, the `-Password` parameter was **not passed at deploy time** — PostgreSQL on both servers has password `unused`
- NiFi is running over plain HTTP; the single-user login is not enforced

Other open issues

- OpenSearch security plugin is disabled (`DISABLE_SECURITY_PLUGIN=true`)
 - NiFi web UI runs over HTTP, bypassing the single-user authenticator
 - Ozone S3G access key and secret are both `tazama`
-

G.2 - Credential rotation (immediate priority)

These are the credentials that need to be changed before the beta is used with any real or sensitive data. Work through each service in order.

PostgreSQL — both servers

The simplest approach for the current deployment is to change the password directly in the running container and patch the env files on disk.

```
cd "full-stack-docker-tazama\infra\aws\scripts"
. .\helpers.ps1
$out = Get-TofuOutputs

$newPgPassword = Read-Host "Enter new PostgreSQL password" -AsSecureString
$pgPw = [Runtime.InteropServices.Marshal]::PtrToStringAuto(
    [Runtime.InteropServices.Marshal]::SecureStringToBSTR($newPgPassword))

# Server A
Invoke-RemoteCommand -InstanceId $out.ServerA_InstanceID -Command `
    "docker exec tazama-core-postgres-1 psql -U postgres -c `\"ALTER USER postgres PASSWORD '$pgPw'`\""

# Server B
Invoke-RemoteCommand -InstanceId $out.ServerB_InstanceID -Command `
    "docker exec tazama-extensions-postgres-1 psql -U postgres -c `\"ALTER USER postgres PASSWORD '$pgPw'`\""
```

Then re-run the deploy scripts with the new password so all service env files are updated:

```
.\deploy-core.ps1 -Password $pgPw -NoPull
.\deploy-extensions.ps1 -Password $pgPw -NoPull
```

-NoPull skips the image pull and only patches the env files and restarts services — substantially faster than a full redeploy.

Long-term: Store the password in SSM as `/tazama/postgres_core_password` and `/tazama/postgres_extensions_password` (see A.7 SSM commands) and pass `-Password` on every future deploy. The deploy scripts already read from SSM if the `-Password` parameter is not supplied directly.

NiFi — enable HTTPS and enforce login

NiFi's single-user authenticator only activates when NiFi is serving over HTTPS. With `NIFI_WEB_HTTP_PORT` set, anyone who reaches port 8088 is in without a password.

Step 1 — Update `biar/env/nifi.env` on Server C:

```
# On Server C (via Invoke-RemoteCommand or SSH)
sed -i 's/NIFI_WEB_HTTP_PORT=.*//' ~/full-stack-docker-tazama/biar/env/nifi.env
sed -i 's/NIFI_WEB_HTTP_HOST=.*//' ~/full-stack-docker-tazama/biar/env/nifi.env
echo "NIFI_WEB_HTTPS_PORT=8443" >> ~/full-stack-docker-tazama/biar/env/nifi.env
echo "NIFI_WEB_HTTPS_HOST=0.0.0.0" >> ~/full-stack-docker-tazama/biar/env/nifi.env
```

Step 2 — Update the `biar` compose file to expose port 8443 instead of 8088, and update the ALB target group to point to 8443. This requires a `tofu apply` to update the ALB listener rule.

Step 3 — Set strong credentials:

```
Invoke-RemoteCommand -InstanceId $out.ServerC_InstanceID -Command `
    "docker exec biar-nifi /opt/nifi/nifi-current/bin/nifi.sh set-single-user-credentials admin '<strong-password>'"
```

NiFi requires a minimum of 12 characters. Store the password in SSM as `/tazama/nifi_admin_password`.

Step 4 — Restart NiFi:


```
Invoke-RemoteCommand -InstanceId $out.ServerC_InstanceId -Command `
"cd ~/full-stack-docker-tazama/biar && docker compose -p tazama-biar \
-f ./docker-compose.biar.infrastructure.yaml \
-f ./docker-compose.hub.biar.yaml \
-f ./docker-compose.utils.init.yaml \
restart nifi"
```

Apache Ozone S3G

```
# On Server C
sed -i 's/OZONE-SITE.XML_ozone.s3g.secret.key=.* /OZONE-SITE.XML_ozone.s3g.secret.key=<strong-secret>/' \
~/full-stack-docker-tazama/biar/env/ozone-docker-config

# Restart s3g
cd ~/full-stack-docker-tazama/biar
docker compose -p tazama-biar \
-f ./docker-compose.biar.infrastructure.yaml \
-f ./docker-compose.hub.biar.yaml \
-f ./docker-compose.utils.init.yaml \
restart s3g
```

Also update `S3A_ACCESS_KEY` and `S3A_SECRET_KEY` in any service that connects to Ozone (e.g. `biar/env/jupyterlab.env`) to match.

Remaining credentials (A.7 full SSM rollout)

The remaining services (Redis, CouchDB, TRS signing key, Hasura, pgAdmin, OAuth client secret, relay auth, CMS Flowable, OpenSearch) are covered by the SSM parameter table in A.7. Once all SSM parameters are populated:

```
# Populate all SSM parameters (see A.7 for the full command block)
# Then redeploy with the -Password flag to trigger the overlay across all services
.\deploy.ps1 -Password $pgPw
```

G.3 - Re-enable OpenSearch security plugin

See A.6. OpenSearch is currently running with `DISABLE_SECURITY_PLUGIN=true`. This is the highest-risk open item after credential rotation because OpenSearch is reachable from Server C (NiFi) and potentially accessible to any service on the private subnet without authentication.

To fix: Remove `DISABLE_SECURITY_PLUGIN=true` from the `opensearch-node1` service in `extensions/docker-compose.extensions.infrastructure.yaml`, set a strong password in SSM as `/tazama/opensearch_password`, and redeploy the extensions stack.

G.4 - Security hardening status summary

Item	Status	Action required
VPC private subnet isolation	✔ Done	—
EICE-only SSH access	✔ Done	—
ALB HTTPS with ACM certificates	✔ Done	—
ALB Keycloak authentication on listeners	✔ Done	—
PostgreSQL passwords rotated	✘ Default (unused)	G.2 — rotate and store in SSM
NiFi login enforced	✘ HTTP, no auth	G.2 — enable HTTPS, set password

Item	Status	Action required
Ozone S3G credentials rotated	✗ Default (tazama/tazama)	G.2 — rotate key/secret
OpenSearch security plugin enabled	✗ Disabled	G.3 — re-enable, set password
Keycloak admin password rotated	✗ Default	A.7 — pass <code>-Password</code> at deploy
Redis/Valkey password set	✗ Default	A.7 SSM rollout
CouchDB password rotated	✗ Default	A.7 SSM rollout
TRS crypto signing key set	✗ Default	A.7 SSM rollout
Hasura admin secret set	✗ Default	A.7 SSM rollout
Auth service OAuth client secret set	✗ Default	A.7 SSM rollout
NiFi HTTPS port on ALB	✗ HTTP 8088	G.2 — compose + tofu update
pgAdmin passwords rotated	✗ Default	A.7 SSM rollout
SFTP password rotated	✗ Default	A.7 — requires compose change first

Troubleshooting

EICE SSH tunnel fails with [AccessDeniedException](#) / bootstrap never completes

Symptom: `deploy-core.ps1` (or any deploy script) keeps printing `still bootstrapping - retrying in 30s` for 15 minutes and then throws a timeout, even though the EC2 instance is running and the bootstrap actually completed. Running SSH manually shows:

```
awscli.customizations.ec2instanceconnect.websocket - ERROR - {"ErrorCode":"AccessDeniedException",
"Message":"User: arn:aws:iam::<account>:user/tazama-deploy is not authorized to perform:
ec2-instance-connect:OpenTunnel on resource: ..."}

```

Cause: `ec2-instance-connect:OpenTunnel` (required for the EICE SSH tunnel) is **not** included in [AmazonEC2FullAccess](#). It is a separate service namespace. Without it, every SSH attempt the deploy scripts make is silently rejected.

Fix: Create and attach a customer-managed policy (see B.6 for the full step, or run this quick version):

```
$policy = '{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Action":["ec2-instance-
connect:OpenTunnel","ec2-instance-connect:SendSSHPublicKey"],"Resource":["*"]}]}
$policy | Out-File "$env:TEMP/eice-policy.json" -Encoding ASCII
$policyArn = (& "C:\Program Files\Amazon\AWSCLI\aws.exe" iam create-policy `
  --policy-name TazamaEICEAccess `
  --policy-document "file://$env:TEMP/eice-policy.json" `
  --profile tazama --query Policy.Arn --output text)
& "C:\Program Files\Amazon\AWSCLI\aws.exe" iam attach-user-policy `
  --user-name tazama-deploy --policy-arn $policyArn --profile tazama

```

Once applied, re-run the deploy script. `Wait-Bootstrap` will now be able to connect and will either find `.bootstrap-complete` immediately (if bootstrap already finished during the failed attempts) or wait until it does.

Permanent fix: B.6 now includes a step to create and attach [TazamaEICEAccess](#) as a customer-managed policy. New deployments will not hit this issue.

Docker Desktop fails with "500 Internal Server Error"

Symptom: `docker ps` returns `error during connect: ... 500 Internal Server Error`

Cause: WSL2 virtual machine enters a bad state (common after sleep/hibernate or resource exhaustion).

Fix:

```
wsl --shutdown
```

Then restart Docker Desktop from the system tray (right-click → Restart). Wait ~30 seconds before retrying Docker commands.

Prevention: Add a `.wslconfig` file at `C:\Users\<you>\.wslconfig` to cap WSL2 memory usage:

```
[wsl2]
memory=8GB
swap=4GB
```

EC2 `RunInstances` fails with `PendingVerification`

Symptom: `tofu apply` fails with:

```
Error: creating EC2 Instance: operation error EC2: RunInstances,
api error PendingVerification: Your request for accessing resources in this region
is being validated, and you will not be able to launch additional resources in this
region until the validation is complete.
```

Cause: AWS activates a region-verification hold the first time an account requests EC2 instances in a region (or after a long period of inactivity). It is an AWS account-level gate, not a code issue. Everything provisioned before the EC2 instances -- VPC, subnets, IGW, NAT GW, EICE endpoint, IAM role, security groups, Route 53 zone -- will have succeeded and is already recorded in OpenTofu state.

Fix:

1. Wait. AWS says "normally resolved within minutes" -- in practice 15-60 minutes for accounts with billing activity. The 4-hour ceiling is the worst case.
2. Check the email address on the AWS account root user. AWS sends a notification when verification completes.
3. Once cleared, re-run plan and apply from the IaC directory. OpenTofu reads the S3 state file, skips everything already created, and plans only the three EC2 instances still missing:

```
cd full-stack-docker-tazama\infra\aws
tofu plan -var-file terraform.tfvars -out tfplan
tofu apply tfplan
```

4. If it has not cleared after 4 hours, open a support case. Use the URL from the error message -- it pre-fills the correct category (`account-management` / `account-verification`).

Note: This hold only triggers once per region per account. Subsequent deployments (e.g. `tofu destroy` followed by `tofu apply`) will not hit it again.

`tofu apply` fails with `AccessDeniedException`: `acm:RequestCertificate`

Symptom: Applying Option 3 (`domain.tfvars`) fails with:

```
| Error: requesting ACM Certificate (*.your-zone): operation error ACM: RequestCertificate,
| api error AccessDeniedException: User: arn:aws:iam::<account>:user/tazama-deploy is not
| authorized to perform: acm:RequestCertificate on resource: arn:aws:acm:...
```

Cause: `acm:RequestCertificate` is not included in any AWS-managed policy. The `tazama-deploy` user needs a customer-managed policy granting ACM access.

Fix: Create and attach the `TazamaACMAccess` policy (see B.6, or run this quick version):

```
$policy = '{"Version":"2012-10-17","Statement":[{"Effect":"Allow","Action":["acm:*"],"Resource":'*"]}'
$policy | Out-File "$env:TEMP\acm-policy.json" -Encoding ASCII
$policyArn = (& "C:\Program Files\Amazon\AWSCLI2\aws.exe" iam create-policy `
  --policy-name TazamaACMAccess `
  --policy-document "file://$env:TEMP/acm-policy.json" `
  --profile tazama --query Policy.Arn --output text)
& "C:\Program Files\Amazon\AWSCLI2\aws.exe" iam attach-user-policy `
  --user-name tazama-deploy --policy-arn $policyArn --profile tazama
```

Once attached, re-run the apply. OpenTofu will skip resources already created and retry the ACM certificate:

```
tofu apply -var-file terraform.tfvars -var-file alb.tfvars -var-file domain.tfvars
```

Permanent fix: B.6 now includes a step to create and attach `TazamaACMAccess`. New deployments will not hit this issue.

`tofu apply` fails with "Module not installed"

Symptom: Running `tofu apply` (with or without `-var-file alb.tfvars`) fails with:

```
Error: Module not installed

on main.tf line 170:
170: module "alb" {

This module is not yet installed. Run "tofu init" to install all modules required by this configuration.
```

Cause: `tofu init` was run before the `module "alb"` and `module "dns_public"` declarations were added to `main.tf`. OpenTofu caches module metadata at init time and does not re-scan automatically.

Fix: Re-run `tofu init` — it is always safe to run against an existing deployment and will not modify state:

```
tofu init
tofu apply -var-file terraform.tfvars -var-file alb.tfvars -var-file domain.tfvars
```

Note: Anyone following the guide from scratch will not hit this. The initial `tofu init` in Phase C picks up all module declarations including `module "alb"` and `module "dns_public"`, even when their `count` is 0. Re-running `init` is only needed if you added these modules to an already-initialised working directory.

ALB health check returns "Blocked request. This host is not allowed."

Symptom: Accessing a service via the ALB DNS name (e.g. `http://<alb-dns>:5000/health`) returns:

```
Blocked request. This host ("<alb-dns>.elb.amazonaws.com") is not allowed.
To allow this host, add "<alb-dns>.elb.amazonaws.com" to `server.allowedHosts` in vite.config.js.
```

Cause: The ALB is routing correctly — this response is coming from the service. The service is a Vite-based frontend, and Vite's built-in host-checking security feature rejects requests where the `Host` header is not an explicitly allowed hostname. When accessed through the ALB, the `Host` header is the ALB DNS name, which Vite does not recognise by default.

Fix: In the service's `vite.config.js` (or `vite.config.ts`), add the ALB hostname to `server.allowedHosts`:

```
server: {
  allowedHosts: ['<alb-dns>.elb.amazonaws.com'],
```

```
}
```

For a sandbox where the ALB DNS name may change between deployments, use `'all'` to disable host checking entirely:

```
server: {
  allowedHosts: 'all',
}
```

Note: `allowedHosts: 'all'` is acceptable for a private developer sandbox. Do not use it in a production deployment — use the explicit hostname list or switch to the custom-domain HTTPS configuration (Phase G), which serves all services through a single domain that can be permanently whitelisted.

Tip: Backend services (those exposing a REST API) will not show this error. It only appears for services that embed a Vite dev server. Use a backend health endpoint to confirm ALB routing independently of the Vite issue.

Accessing container logs on an EC2 instance

1. Set up the `.ssh` folder and copy your key

The `.ssh` folder is not created automatically on Windows — create it and copy the EC2 key into it:

```
# Create folder (safe to run even if it already exists)
New-Item -ItemType Directory -Force -Path "$env:USERPROFILE\.ssh"

# Restrict permissions (SSH client requires this)
icacls "$env:USERPROFILE\.ssh" /inheritance:r /grant "${env:USERNAME}:(OI)(CI)F"

# Copy key from its location in the repo
Copy-Item "<path-to-repo>\full-stack-docker-tazama\infra\aws\tazama-aws.pem"
"$env:USERPROFILE\.ssh\tazama-aws.pem"
```

2. Create the SSH config file

On Windows the SSH config file is at `C:\Users\<your-username>\.ssh\config` (create it if it does not exist):

```
New-Item -ItemType File -Force -Path "$env:USERPROFILE\.ssh\config"
notepad "$env:USERPROFILE\.ssh\config"
```

Add an entry for each server (replace instance IDs with the values from `tofu output`):

```
tofu -chdir=full-stack-docker-tazama/infra/aws output -json | ConvertFrom-Json | Select-Object
server_a_instance_id, server_b_instance_id, server_c_instance_id
```

```
Host tazama-a
  HostName i-0abc123
  User ec2-user
  IdentityFile ~/.ssh/tazama-aws.pem
  ProxyCommand aws ec2-instance-connect open-tunnel --instance-id %h --remote-port 22 --region ap-south-1
  --profile tazama

Host tazama-b
  HostName i-0def456
  User ec2-user
  IdentityFile ~/.ssh/tazama-aws.pem
  ProxyCommand aws ec2-instance-connect open-tunnel --instance-id %h --remote-port 22 --region ap-south-1
  --profile tazama
```

```
Host tazama-c
  HostName i-0ghi789
  User ec2-user
  IdentityFile ~/.ssh/tazama-aws.pem
  ProxyCommand aws ec2-instance-connect open-tunnel --instance-id %h --remote-port 22 --region ap-south-1
  --profile tazama
```

Then connect with just:

```
ssh tazama-a
```

3. View container logs

The Tazama stacks are started with multiple `-f` flags, so `docker compose logs` requires knowing all the compose files used. The simpler approach is to use the container name directly — it works regardless of directory or how the stack was started.

First, list all running containers to find the name:

```
# All containers, names only
docker ps --format '{{.Names}}'

# Filter by service (e.g. keycloak)
docker ps --format '{{.Names}}' | grep keycloak
```

Container names follow the pattern `<project>-<service>-<number>`, e.g. `tazama-core-keycloak-1`.

Then view logs using the container name:

```
# Tail live logs
docker logs -f tazama-core-keycloak-1

# Last N lines only
docker logs --tail=100 tazama-core-keycloak-1

# Logs since a specific time
docker logs --since="2026-04-15T10:00:00" tazama-core-keycloak-1
```

Reference: Compose Chain Matrix (Server A)

Extracted from `tazama-core.bat.docker-compose.base.override.yaml` is always **position 2** - it publishes the three exterior ports that cross-stack services depend on: NATS :14222, PostgreSQL :15432, Valkey :16379.

File	hub	dev	full	multi
docker-compose.base.infrastructure.yaml	✓	✓	✓	✓
docker-compose.base.override.yaml	✓	✓	✓	✓
docker-compose.hub.cfg.yaml	✓	-	-	-
docker-compose.full.cfg.yaml	-	-	✓	-
docker-compose.multitenant.cfg.yaml	-	-	-	✓
docker-compose.dev.cfg.yaml	-	✓	-	-
docker-compose.hub.core.yaml	✓	-	✓	✓
docker-compose.dev.core.yaml	-	✓	-	-

File	hub	dev	full	multi
docker-compose.hub.rules.yaml	✓	-	-	✓
docker-compose.full.rules.yaml	-	-	✓	-
docker-compose.base.auth.yaml	+auth	+auth	+auth	✓
docker-compose.dev.auth.yaml	-	+auth	-	-
docker-compose.hub.relay.yaml	+relay	-	+relay	-
docker-compose.multitenant.relay.yaml	-	-	-	✓
docker-compose.dev.relay.yaml	-	+relay	-	-
docker-compose.hub.logs.base.yaml	+logs	-	+logs	+logs
docker-compose.dev.logs.base.yaml	-	+logs	-	-
docker-compose.utils.pgadmin.yaml	+pgadmin	+pgadmin	+pgadmin	+pgadmin
docker-compose.utils.hasura.yaml	+hasura	+hasura	+hasura	+hasura

AWS beta sandbox - DockerHub images (hub):

```
docker compose -p tazama-core \
  -f ./docker-compose.base.infrastructure.yaml \
  -f ./docker-compose.base.override.yaml \
  -f ./docker-compose.hub.cfg.yaml \
  -f ./docker-compose.hub.core.yaml \
  -f ./docker-compose.hub.rules.yaml \
  -f ./docker-compose.base.auth.yaml \
  -f ./docker-compose.hub.relay.yaml \
  -f ./docker-compose.hub.logs.base.yaml \
  -f ./docker-compose.utils.pgadmin.yaml \
  -f ./docker-compose.utils.hasura.yaml \
  up -d
```

AWS beta sandbox - GitHub source build fallback (dev):

```
docker compose -p tazama-core \
  -f ./docker-compose.base.infrastructure.yaml \
  -f ./docker-compose.base.override.yaml \
  -f ./docker-compose.dev.cfg.yaml \
  -f ./docker-compose.dev.core.yaml \
  -f ./docker-compose.base.auth.yaml \
  -f ./docker-compose.dev.auth.yaml \
  -f ./docker-compose.dev.relay.yaml \
  -f ./docker-compose.dev.logs.base.yaml \
  -f ./docker-compose.utils.pgadmin.yaml \
  -f ./docker-compose.utils.hasura.yaml \
  up -d
```

Reference: Port Map

Server A (tazama-core) - exterior ports

Port	Service	Used by	Subdomain (*.beta.tazama.org)
5000	TMS API	ALB, Postman	tms
3001	DEAPI	ALB, Server B	deapi
3002	DEMS	ALB, Server B	dems

Port	Service	Used by	Subdomain (*.beta.tazama.org)
3020	Auth Service	Server B (JWT validation)	auth
5100	Admin API	Internal	admin
8080	Keycloak	ALB, frontends	keycloak
14222	NATS	Server B relay	—
15432	PostgreSQL	Server C NiFi ETL	—
16379	Valkey	—	—
5050	pgAdmin	Operator (EICE only)	pgadmin
6100	Hasura	Operator (EICE only)	hasura

Server B (tazama-extensions) - exterior ports

Port	Service	Used by	Subdomain (*.beta.tazama.org)
3005	TRS Backend	ALB	trs-api
3010	TCS Backend	ALB	tcs-api
3090	CMS Backend	ALB	cms-api
5173	TCS Frontend	ALB	tcs
5174	TRS Frontend	ALB	trs
5175	CMS Frontend	ALB	cms
9200	OpenSearch	(NiFi ETL - pending confirmation)	—
12222	SFTP	File ingest	—
15433	PostgreSQL (CMS)	Server C NiFi ETL	—

Server C (tazama-biar) - exterior ports

Port	Service	Used by	Subdomain (*.beta.tazama.org)
7619	Automation Orchestrator API	ALB, Operator	automation-orchestrator
8000	JupyterHub	ALB	jupyter
8088	NiFi UI	ALB	nifi
8282	Datalakehouse API	ALB, Operator	datalakehouse-api
8983	Solr UI	Operator (EICE only)	—
9876	Ozone SCM	Operator (EICE only)	—
9878	Ozone S3G	Operator (EICE only)	—
9888	Ozone Recon UI	Operator (EICE only)	—
9998	Tika	Internal only	—

Frequently Asked Questions

I've set up some users on Keycloak already — how can I save these if I want to redeploy the system somewhere else or in the future?

Export the live realm (including users) from the running Keycloak container, copy it to your local machine, and commit it to the repo. The next deploy will import it automatically on first boot.

Step 1 — Export the realm inside the container on Server A:


```
cd "full-stack-docker-tazama\infra\aws\scripts"
. .\helpers.ps1
$out = Get-TofuOutputs

# Run the Keycloak export tool inside the running container
Invoke-RemoteCommand -InstanceId $out.ServerA_InstanceID -Command `
  "docker exec tazama-core-keycloak-1 /opt/keycloak/bin/kc.sh export --realm tazama --dir /tmp --users
  realm_file"

# Copy the export out of the container onto the EC2 host filesystem
Invoke-RemoteCommand -InstanceId $out.ServerA_InstanceID -Command `
  "docker cp tazama-core-keycloak-1:/tmp/tazama-realm.json /home/ec2-user/tazama-realm.json"
```

Step 2 — SCP the file from Server A to your local machine:

```
$configPath = New-SshConfig -InstanceId $out.ServerA_InstanceID
scp -F $configPath "$($out.ServerA_InstanceID):/home/ec2-user/tazama-realm.json" `
  "full-stack-docker-tazama\core\auth\keycloak\realms\00-tazama-test-realm.json"
Remove-Item $configPath
```

Step 3 — Commit and push:

```
cd "full-stack-docker-tazama"
git add core/auth/keycloak/realms/00-tazama-test-realm.json
git commit -S -s -m "chore: export live Keycloak tazama realm"
git push origin <your-branch>
```

The updated realm JSON will be copied to the server by `deploy-core.ps1` and imported by Keycloak on its next fresh start (`--import-realm` only runs when the realm does not already exist in Keycloak's database — see the note below).

Important: Keycloak only imports a realm JSON on first boot if the realm is not already present in its database. If the container is simply restarted without wiping its data, the import is skipped. To force a reimport of an updated realm JSON, the Keycloak container and its embedded H2 database must be removed first:

```
Invoke-RemoteCommand -InstanceId $out.ServerA_InstanceID -Command "docker rm -f tazama-core-
keycloak-1"
. \deploy-core.ps1 -NoPull
```

How do I give another user access to the servers via SSH to allow them to view the docker container logs?

Access to the EC2 instances uses AWS EC2 Instance Connect Endpoint (EICE) — there is no open port 22, and no shared key pair. Each user authenticates with their own AWS IAM identity. To grant another user SSH access you need to give their IAM identity permission to use EICE, and then add their SSH public key to the EC2 instance so the server accepts their connection.

Step 1 — Ensure the user has an IAM identity in the Tazama AWS account

The user must have an IAM user in the AWS account. If they do not have one yet, create it as described in Step 2c below.

Step 2 — Create an IAM group with the required EICE permissions (one-time setup)

AWS recommends attaching permissions to a group rather than directly to individual users. Create a dedicated group for Tazama server operators and attach the EICE policy to it. This only needs to be done once — future users are onboarded by adding them to the group.

2a — Create the group:

In the AWS Console go to **IAM** → **User groups** → **Create group**. Name it `tazama-server-operators` (or similar).

2b — Attach a permissions policy to the group:

In the group, go to **Permissions** → **Add permissions** → **Create inline policy**, paste the following, and name the policy `tazama-server-operators-policy`:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowEICETunnel",
      "Effect": "Allow",
      "Action": [
        "ec2-instance-connect:OpenTunnel",
        "ec2-instance-connect:SendSSHPublicKey"
      ],
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "ec2:ResourceTag/Name": ["tazama-server-a", "tazama-server-b", "tazama-server-c"]
        }
      }
    },
    {
      "Sid": "AllowDescribeForEICE",
      "Effect": "Allow",
      "Action": [
        "ec2:DescribeInstances",
        "ec2:DescribeInstanceConnectEndpoints"
      ],
      "Resource": "*"
    }
  ]
}
```

`ec2-instance-connect` actions do not support account-scoped ARNs in the resource field — `"*"` is the correct and AWS-documented form. The `Condition` block scopes tunnel access to only the three Tazama servers by their `Name` tag, which the OpenTofu configuration sets automatically.

To restrict a user to a subset of servers, simply remove the unwanted server names from the array. For example, to grant access to Server B and C only, remove `"tazama-server-a"` from the list.

2c — Create the IAM user:

In the AWS Console go to **IAM** → **Users** → **Create user**.

- **User name:** use something identifiable, e.g. `firstname.lastname`
- **Provide user access to the AWS Management Console:** leave **unchecked** — they only need CLI access, not console access
- **Permissions:** do not attach any policies here; permissions will come from the group
- Once the user is created, open the user and go to **Security credentials** → **Access keys** → **Create access key**
- Select **Command Line Interface (CLI)** as the use case and complete the wizard
- Download or copy the **Access key ID** and **Secret access key** — these are shown only once
- Send the credentials to the user securely; they will run `aws configure` on their local machine and enter these values

This creates a user with no console access, no permissions outside the group, and credentials scoped only to CLI use.

2d — Add the user to the group:

In the AWS Console go to **IAM** → **User groups** → **tazama-server-operators** → **Users** → **Add users** and select the user. Their permissions are now inherited from the group. To revoke access later, remove them from the group without touching the policy.

Step 3 — User: install and configure the AWS CLI

Send the user their **Access key ID** and **Secret access key** along with the following instructions.

1. Install the AWS CLI (if not already installed):

- Windows: download and run the installer from <https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html>
- Verify: `aws --version`

2. Configure credentials:

```
aws configure
```

Enter the values when prompted:

Prompt	Value
AWS Access Key ID	<i>(as provided)</i>
AWS Secret Access Key	<i>(as provided)</i>
Default region name	<code>ap-south-1</code>
Default output format	<code>json</code>

3. Verify access:

```
aws ec2 describe-instances --query "Reservations[].Instances[].Tags[?Key=='Name'].Value" --output text
```

This should return the names of the three Tazama servers. If it returns an error, the credentials or region are incorrect.

Step 4 — Generate or collect the user's SSH key pair

The user must have an SSH key pair. If they do not have one, they can generate one on their local machine:

```
# On the user's local Windows machine (PowerShell)
ssh-keygen -t ed25519 -C "their-name@example.com" -f "$env:USERPROFILE\.ssh\tazama_ed25519"
```

This creates a private key (`tazama_ed25519`) and a public key (`tazama_ed25519.pub`). They must keep the private key secure and share only the public key with you.

Step 4 — Add the user's public key to the EC2 instance(s)

From your own workstation (which already has access), copy their public key into the `~/.ssh/authorized_keys` file on each instance they need to access.

```
cd "full-stack-docker-tazama\infra\aws\scripts"
. .\helpers.ps1
$out = Get-TofuOutputs

# Paste the full contents of the user's tazama_ed25519.pub file here
# It is a single line starting with "ssh-ed25519 AAAA..."
$pubKey = "ssh-ed25519 AAAA... their-name@example.com"

# Add to Server A
Invoke-RemoteCommand -InstanceId $out.ServerA_InstanceID -Command `
    "echo '$pubKey' >> ~/.ssh/authorized_keys"

# Add to Server B (if needed)
Invoke-RemoteCommand -InstanceId $out.ServerB_InstanceID -Command `
    "echo '$pubKey' >> ~/.ssh/authorized_keys"

# Add to Server C (if needed)
```

```
Invoke-RemoteCommand -InstanceId $out.ServerC_InstanceId -Command `
"echo '$pubKey' >> ~/.ssh/authorized_keys"
```

Step 5 — User: set up the SSH config shortcut

The easiest way to connect is via an SSH config file. This lets the user run `ssh tazama-a` (or `tazama-b` / `tazama-c`) directly without specifying flags each time.

1. Get the instance IDs (run this yourself and share the values with the user):

```
tofu -chdir=full-stack-docker-tazama/infra/aws output -json | ConvertFrom-Json | Select-Object
server_a_instance_id, server_b_instance_id, server_c_instance_id
```

2. User: create or open the SSH config file:

```
New-Item -ItemType File -Force -Path "$env:USERPROFILE\.ssh\config"
notepad "$env:USERPROFILE\.ssh\config"
```

3. User: add an entry for each server they have access to (replace instance IDs with the values you provided):

```
Host tazama-a
  HostName i-0abc123
  User ec2-user
  IdentityFile ~/.ssh/tazama_ed25519
  ProxyCommand aws ec2-instance-connect open-tunnel --instance-id %h --remote-port 22 --region ap-south-1

Host tazama-b
  HostName i-0def456
  User ec2-user
  IdentityFile ~/.ssh/tazama_ed25519
  ProxyCommand aws ec2-instance-connect open-tunnel --instance-id %h --remote-port 22 --region ap-south-1

Host tazama-c
  HostName i-0ghi789
  User ec2-user
  IdentityFile ~/.ssh/tazama_ed25519
  ProxyCommand aws ec2-instance-connect open-tunnel --instance-id %h --remote-port 22 --region ap-south-1
```

Note there is no `--profile` flag here — the user configured a default profile in Step 3, so AWS CLI picks it up automatically.

4. User: connect:

```
ssh tazama-a
```

Step 6 — Viewing Docker container logs

Once connected via SSH, the user can view logs for any running container:

```
# List all running containers
docker ps

# Follow live logs for a specific container (Ctrl+C to stop)
docker logs -f <container-name>

# Show the last 100 lines then follow
docker logs --tail 100 -f <container-name>
```

```
# View logs for all containers in the tazama-core stack
docker compose -p tazama-core logs -f

# View logs for a specific service within the stack
docker compose -p tazama-core logs -f tms-service
```

Common container names in the core stack follow the pattern `tazama-core-<service>-1`, e.g. `tazama-core-tms-service-1`, `tazama-core-keycloak-1`, `tazama-core-arango-1`.

Note: The `ec2-user` account on the instances is in the `docker` group, so `sudo` is not required for `docker` commands.

How do I test if my Data Lakehouse is properly configured and working via JupyterHub?

Log into JupyterHub and open a new notebook. Run the following cells in order:

Cell 1 — Check environment variables:

```
import os

print("WAREHOUSE_ROOT:", os.environ.get("WAREHOUSE_ROOT", "NOT SET"))
print("SPARK_JARS:      ", os.environ.get("SPARK_JARS", "NOT SET"))
print("SPARK_HOME:      ", os.environ.get("SPARK_HOME", "NOT SET"))
print("JAVA_HOME:       ", os.environ.get("JAVA_HOME", "NOT SET"))
```

Expected: all four should show paths, not `NOT SET`. If any paths are missing, check `biar/env/jupyterlab.env` on Server C and confirm it has been updated to `/opt/Tazama_Warehouse`, then restart the container.

Cell 2 — Check the warehouse directory:

```
import os

warehouse = os.environ.get("WAREHOUSE_ROOT", "/opt/Tazama_Warehouse")
tables = os.listdir(warehouse)
print(f"Tables in {warehouse}:")
for t in sorted(tables):
    print(" ", t)
```

Expected: a list of Hudi table directories (e.g. `gold/`, `silver/`, `views/`). If you get a `FileNotFoundError`, the warehouse volume is not mounted — check the `docker-compose.hub.biar.yaml` volume entry and re-run `docker compose up -d jupyterhub`.

Cell 3 — Start a Spark session with the Hudi JAR:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .config("spark.jars", os.environ["SPARK_JARS"]) \
    .config("spark.serializer", "org.apache.spark.serializer.KryoSerializer") \
    .getOrCreate()

print("Spark version:", spark.version)
print("Session started OK")
```

Expected: Spark version printed without errors. A `ClassNotFoundException: hudi.DefaultSource` means the JAR path in `SPARK_JARS` is wrong — verify with `os.listdir("/opt/jars")`.

Cell 4 — Read a Hudi table:

```
# Replace with an actual table path from Cell 2 output, e.g. "gold/transactions"
table_path = os.environ["WAREHOUSE_ROOT"] + "/gold/transactions"

df = spark.read.format("hudi").load(table_path)
print(f"Row count: {df.count()}")
df.printSchema()
```

Expected: schema printed, row count > 0. A `FileNotFoundException` means the path doesn't exist — use the exact directory names from Cell 2. A `DataSourceNotFoundException` means the Hudi JAR did not load — re-check Cell 3.

Cell 5 — Stop the session when done:

```
spark.stop()
print("Spark stopped.")
```

How can I access the JupyterHub server from VS Code?

VS Code's Jupyter extension supports connecting to remote Jupyter servers directly, so you can run notebooks from VS Code using the kernel running on the JupyterHub instance — no need to upload notebooks to the server UI.

Steps:

1. **Generate an API token** on the server

Browse to <https://jupyter.beta.tazama.org/hub/token>, log in as admin, and create a token. Copy it.

2. **Make sure your server is spawned**

Log into <https://jupyter.beta.tazama.org> once so the single-user server starts. VS Code connects to an already-running server — it cannot spawn one for you.

3. **In VS Code, open the notebook and select a kernel**

Click the kernel picker (top-right, shows "Select Kernel") → **Existing Jupyter Server...** → **Enter the URL of the running Jupyter server.**

4. **Enter the server URL**

```
https://jupyter.beta.tazama.org/user/admin/
```

When prompted for a password/token, paste the token from step 1.

5. **Select the Python kernel** from the list that appears — it will be the kernel running inside the `biar-jupyterhub` container on Server C.

Result: VS Code is the UI; the kernel (Spark, Hudi JARs, warehouse mount) runs on Server C. Cell outputs, variables, and plots come back to VS Code. The notebook file stays local on your machine.

Caveats:

- If JupyterHub idles out and shuts down the single-user server, VS Code will lose the connection. Re-spawn via the browser and reconnect.
- Tokens have a lifetime — if yours expires, generate a new one from `/hub/token`.
- The token auth bypasses the browser SSO flow, so the ALB Keycloak listener does not interfere with the `/user/<username>/api` path.